

MPLS

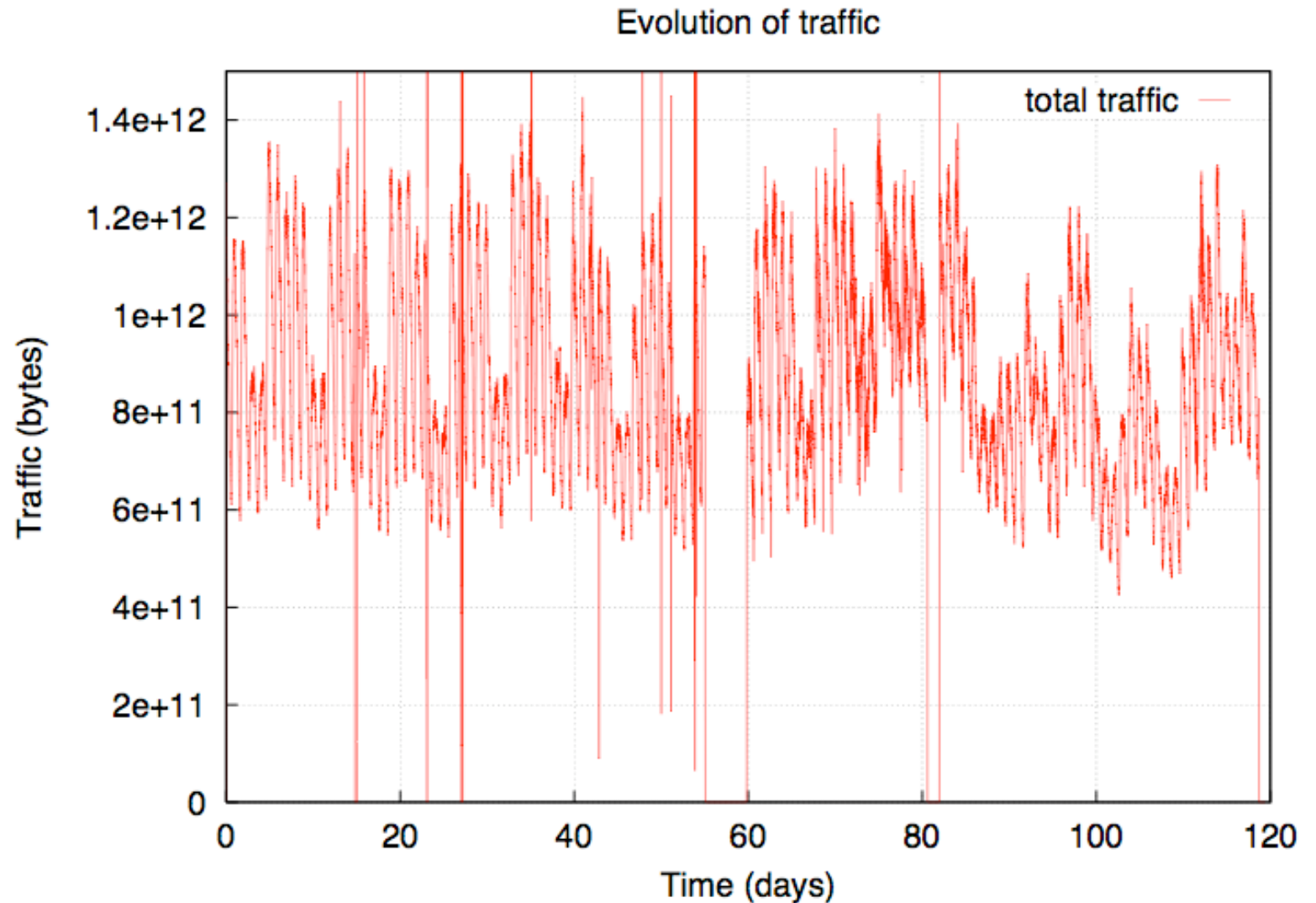
MultiProtocol Label Switching

- Outline
- Multiprotocol Label Switching
 - The label swapping forwarding paradigm
 - Integrating label swapping and IP
- Utilisation of MPLS
 - Destination based packet forwarding
 - Simpler ISP backbones
 - Traffic engineering

[illegible]

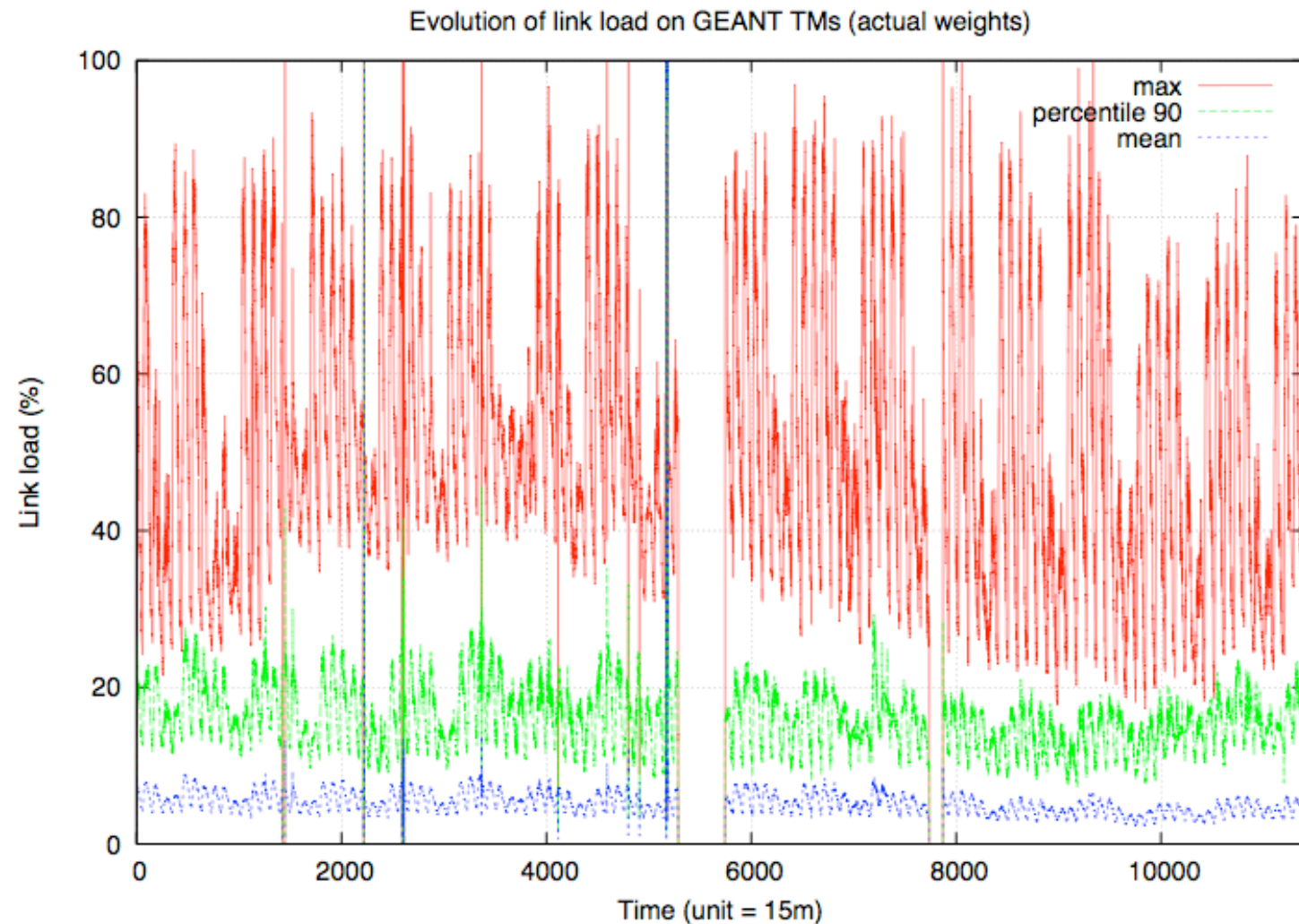
Evolution of traffic load

- Example with GEANT



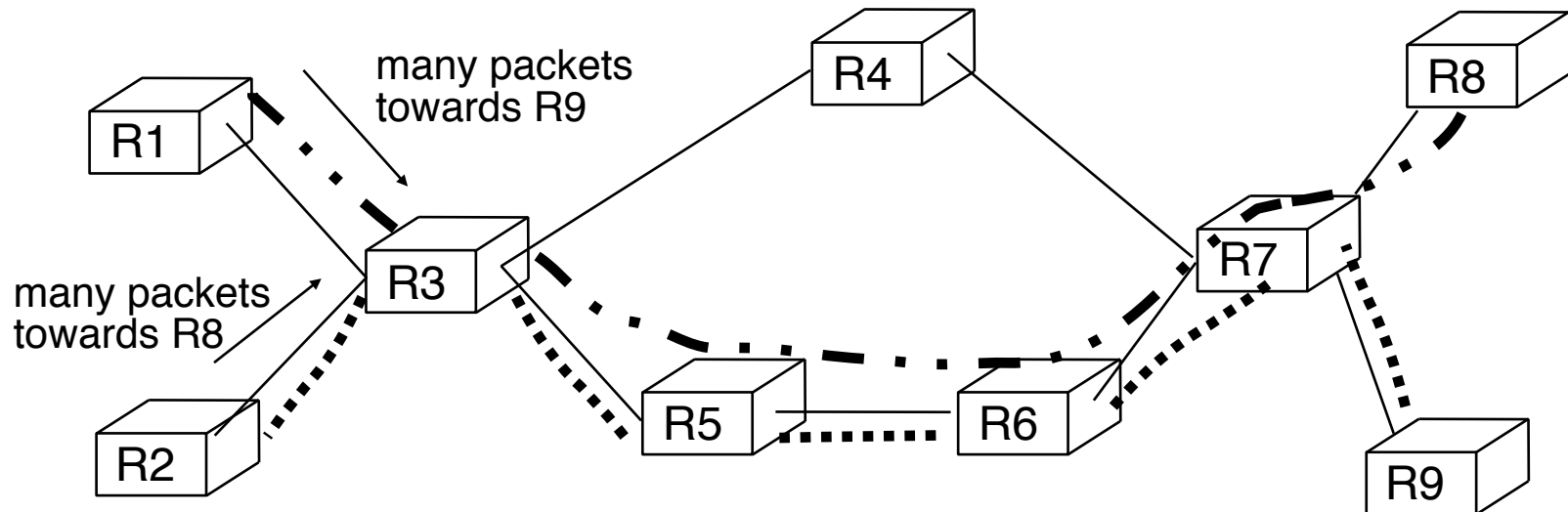
Evolution of link load

- Example with GEANT



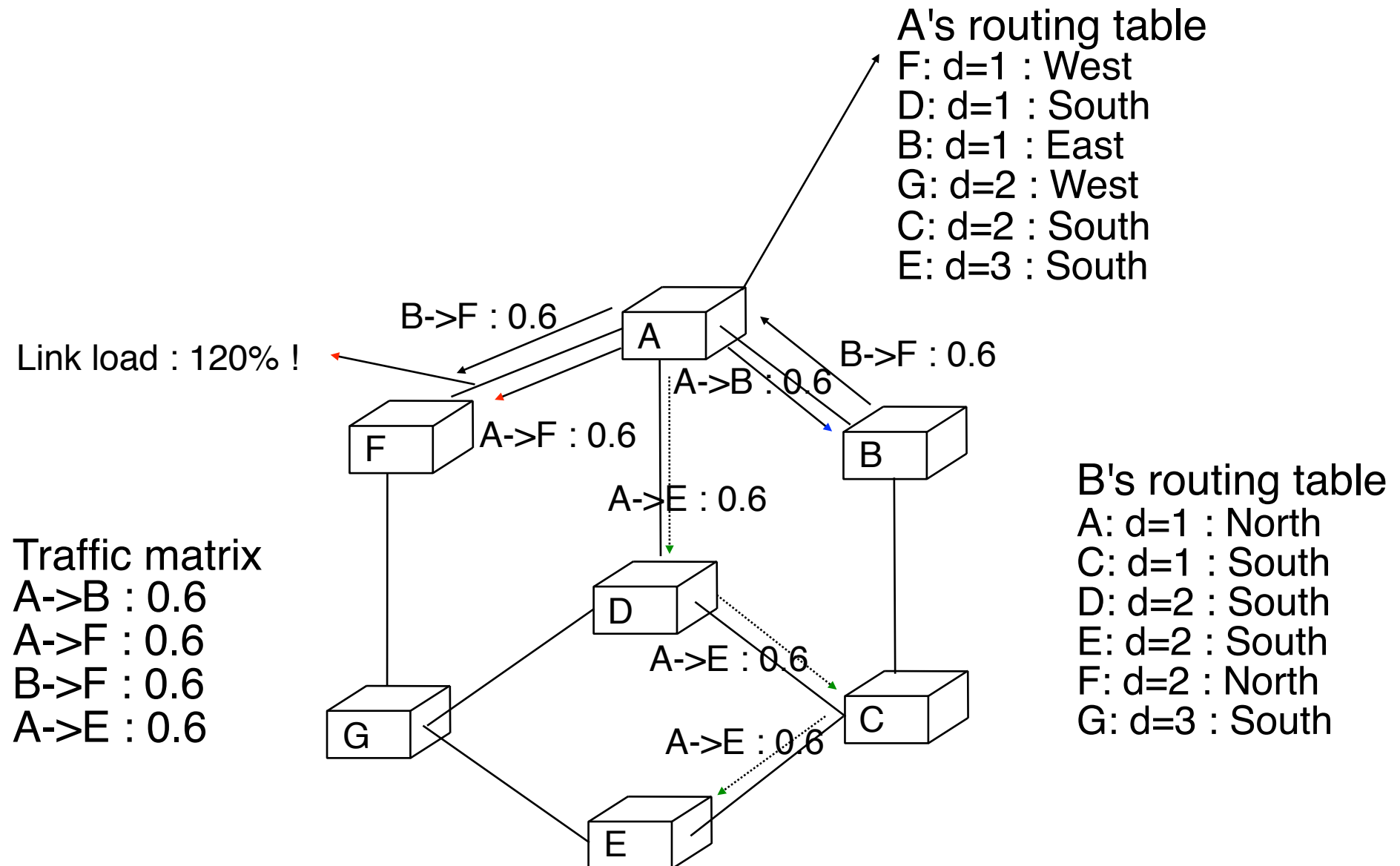
Traffic engineering

- Problem
 - Shortest path chosen by IP routing does not always lead to a good network utilisation
 - fish problem



- How to better optimise the network utilisation ?
- How to react to changes in traffic conditions

Traffic engineering simple case study



IP based traffic engineering

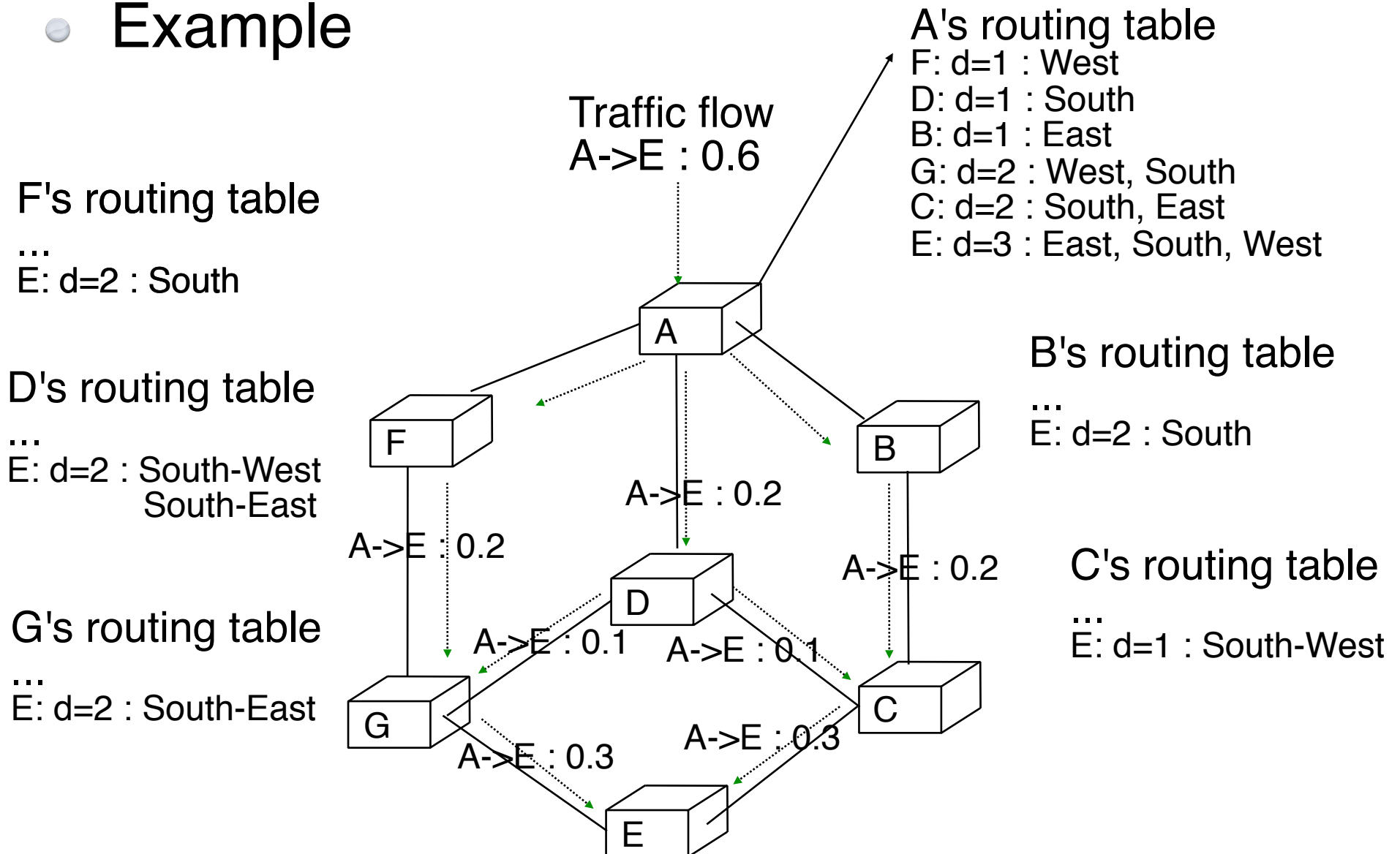
- How to solve the traffic engineering problem in a pure IP network ?
 - Two types of solutions
- Router-level traffic engineering
 - Allow a router to use several paths instead of a single one for a given route
 - Possible with most router implementations
- Network-level traffic engineering
 - Force aggregate traffic flows to follow some paths inside the network
 - Possible in some cases by playing with link costs

Equal Cost Multipath

- Simple network-level load balancing mechanism supported in OSPFv2
 - Principle
 - OSPF distributes the complete network topology to all routers inside network
 - based on this topology, each router computes the routes towards all destinations
 - if a router finds several equal cost paths reaching one destination, it may balance its traffic over these paths
 - load balancing is done at the discretion of this router without coordination with other routers
 - since routes are equal cost routes, loops will not occur provided that the routing table is stable

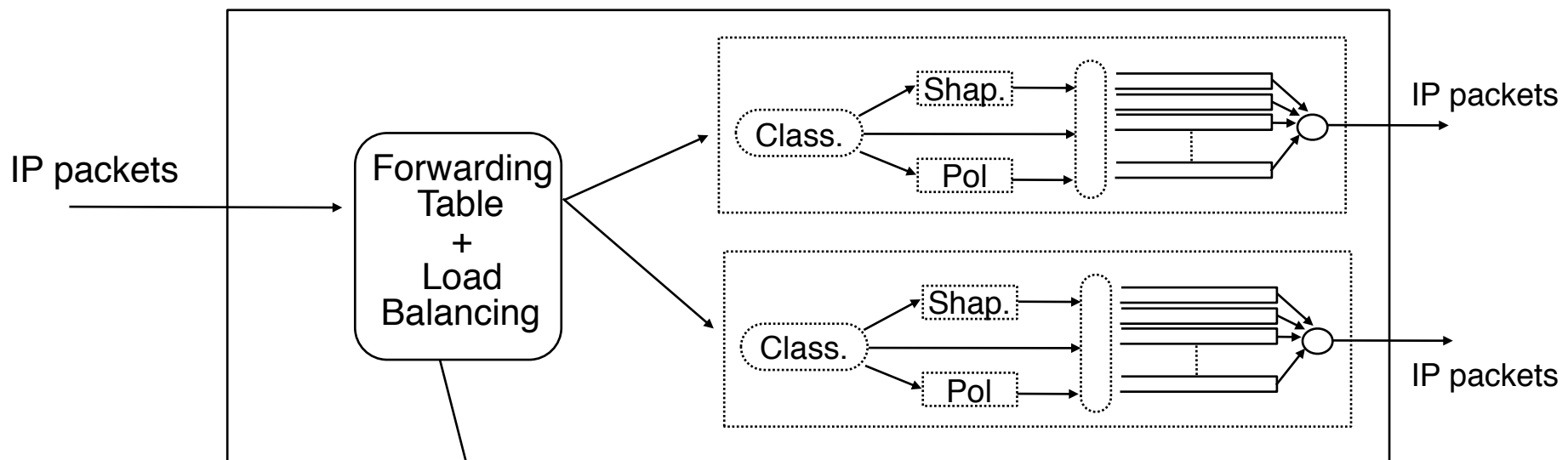
Equal Cost Multipath (2)

• Example



How to dispatch IP packets ?

- Principle
 - For each destination, remember the P equal paths instead of a single one
 - place those paths in forwarding table
 - when a packet arrives, load balancing algorithm selects one path among the P available paths



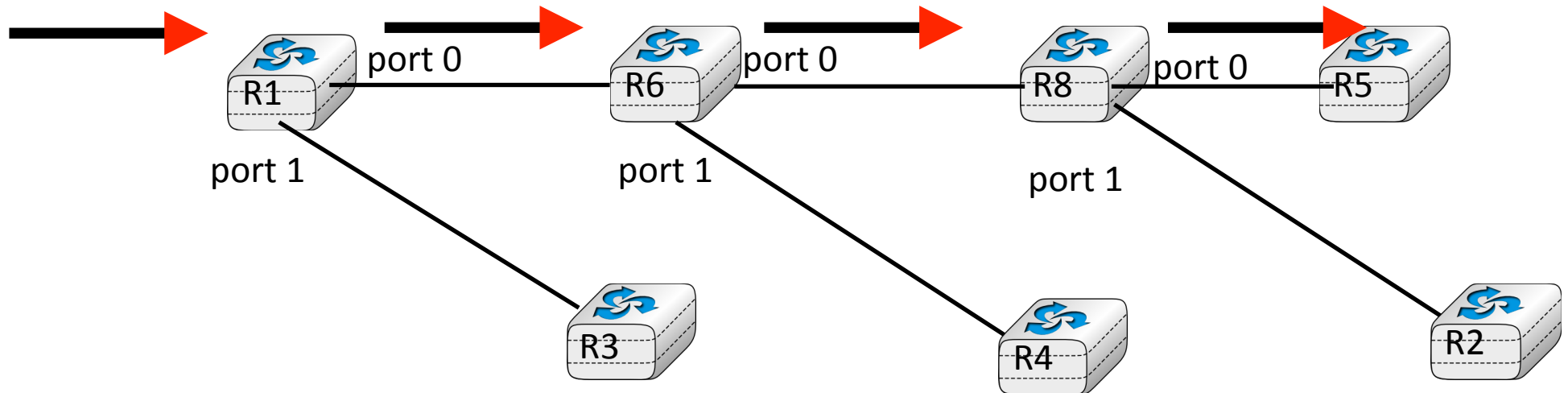
Forwarding decision based on *longest* match
Load balancing implies that forwarding table may contain several entries for each destination

Per TCP connection load balancing

- How to perform load balancing without maintaining state for each TCP connection ?
 - Principle
 - concatenate IP src, IP dest, IP protocol, Src port, and Dest port from the IP packet inside a bit string
 - `bitstring = [IP src:IP dest:IP protocol:Src port:Dest port]`
 - `compute path = Hash(bitstring) mod P`
 - hash function should be easy to implement and should produce very different numbers for close bitstring values
 - candidate hash functions are CRC, checksum, ...
 - Advantages
 - all packets from TCP connection sent on same path
 - traffic to a server will be divided over the links
 - Drawback
 - does not work well if a few TCP connections carry a large fraction of the total traffic
 - Polarisation issues if all routers use same hash

The polarisation problem

- How to prevent this polarisation problem ?



Limitations of Equal Cost Multipath

Traffic matrix

A->B : 0.6

A->F : 0.6

B->F : 0.6

A->E : 0.6

A->E : 0.2

B->F : 0.6

A's routing table

F: d=1 : West

D: d=1 : South

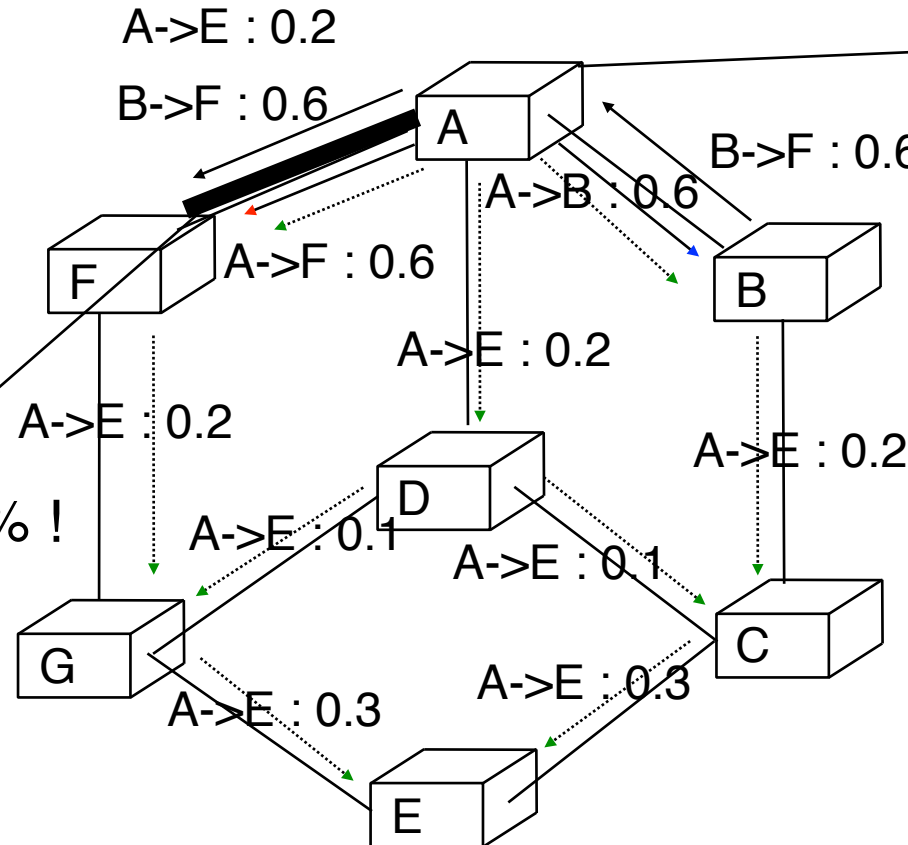
B: d=1 : East

G: d=2 : West, South

C: d=2 : South, East

E: d=3 : East, South, West

Link load : 140% !



- Drawbacks of ECMP
 - load balancing only works for *exactly equal* costs paths and few paths are exactly equal
 - local decision taken by each individual router

IP-based network level traffic engineering

- How to improve the traffic distribution throughout the entire network ?
 - Principle
 - IGP link cost influences the utilisation of this link
 - Typical IGP link cost settings include
 - 1 for each link to select shortest path measured in hops
 - =link delay to select shortest path measured in seconds
 - f(bandwidth) to select shortest-high bandwidth path
 - example :
$$\frac{M}{link\ bandwidth}$$
 - Careful selection of the IGP link costs to balance traffic
 - rerouting traffic outside a busy link by manually tweaking costs
 - optimising the flow of traffic instead a network for a given traffic matrix can considering it as a classical optimisation problem
 - Can be difficult if routers do not support ECM
 - possible with some restrictions when routers support ECM

IP-based network level traffic engineering (2)

Traffic matrix

A→B : 0.6

A→F : 0.6

B→F : 0.6

A→E : 0.6

A→E : 0.2

B→F : 0.6

A's routing table

F: d=1 : West

D: d=1 : South

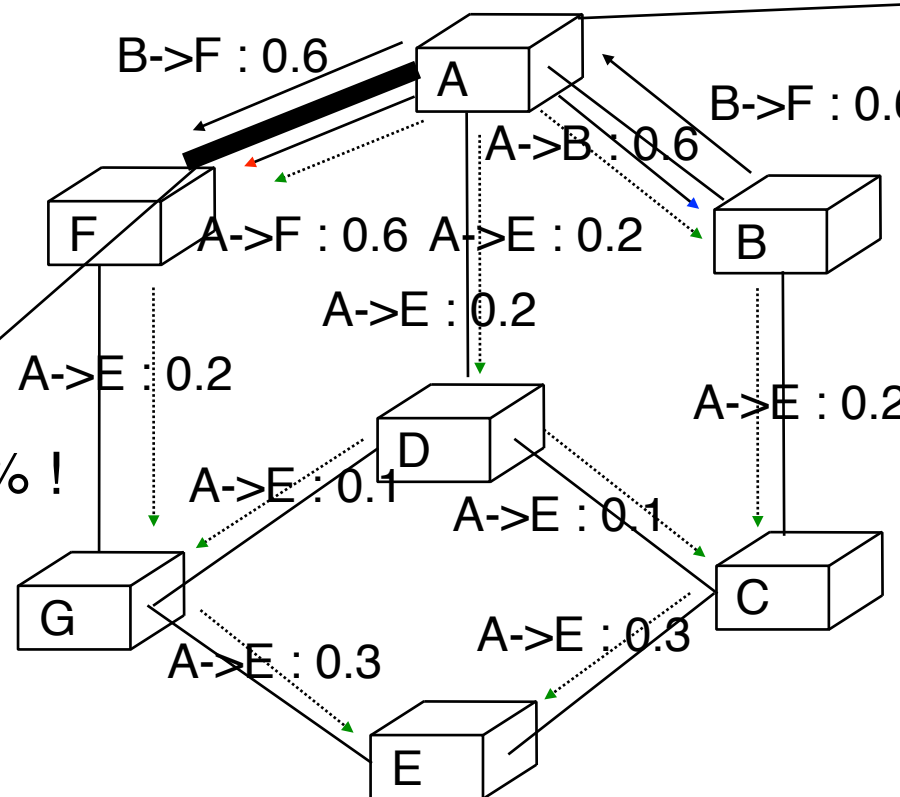
B: d=1 : East

G: d=2 : West, South

C: d=2 : South, East

E: d=3 : East, South, West

Link load : 140% !



- How to improve the traffic distribution ?
 - A should send traffic towards E only via its South port
 - B should send traffic towards F only via its South port
 - Possible by changing the IGP link costs

IP-based network level traffic engineering (3)

- Possible setting of the IGP link costs

D's routing table

A: d=2: North
B: d=2: South-East
C: d=1: South-East
E: d=2: S-E, S-W
F: d=2: S-W
G: d=1: S-W

A's routing table

F: d=3: West
D: d=2: South
B: d=3: East
E: d=4: South
G: d=3: South
C: d=3: South

B's routing table

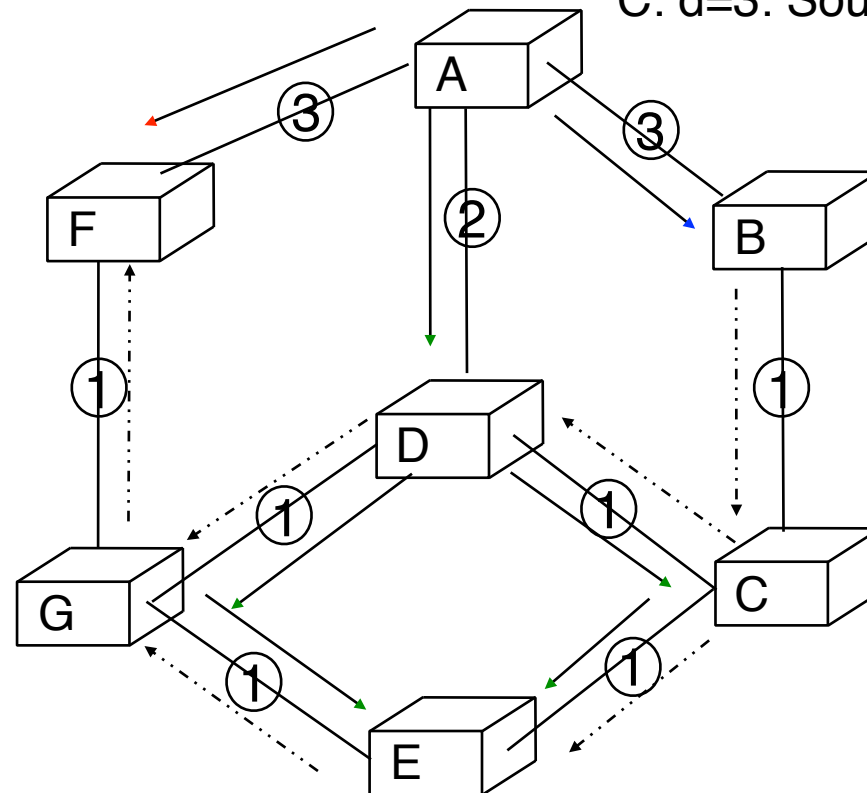
A: d=3: North
C: d=1: South
D: d=2: South
E: d=E: South
F: d=4: South
G: d=3: South

C's routing table

B: d=1: North
A: d=3: North-West
D: d=1: North-West
E: d=1: South-West
F: d=3: N-W, S-W
G: d=2: N-W, S-W

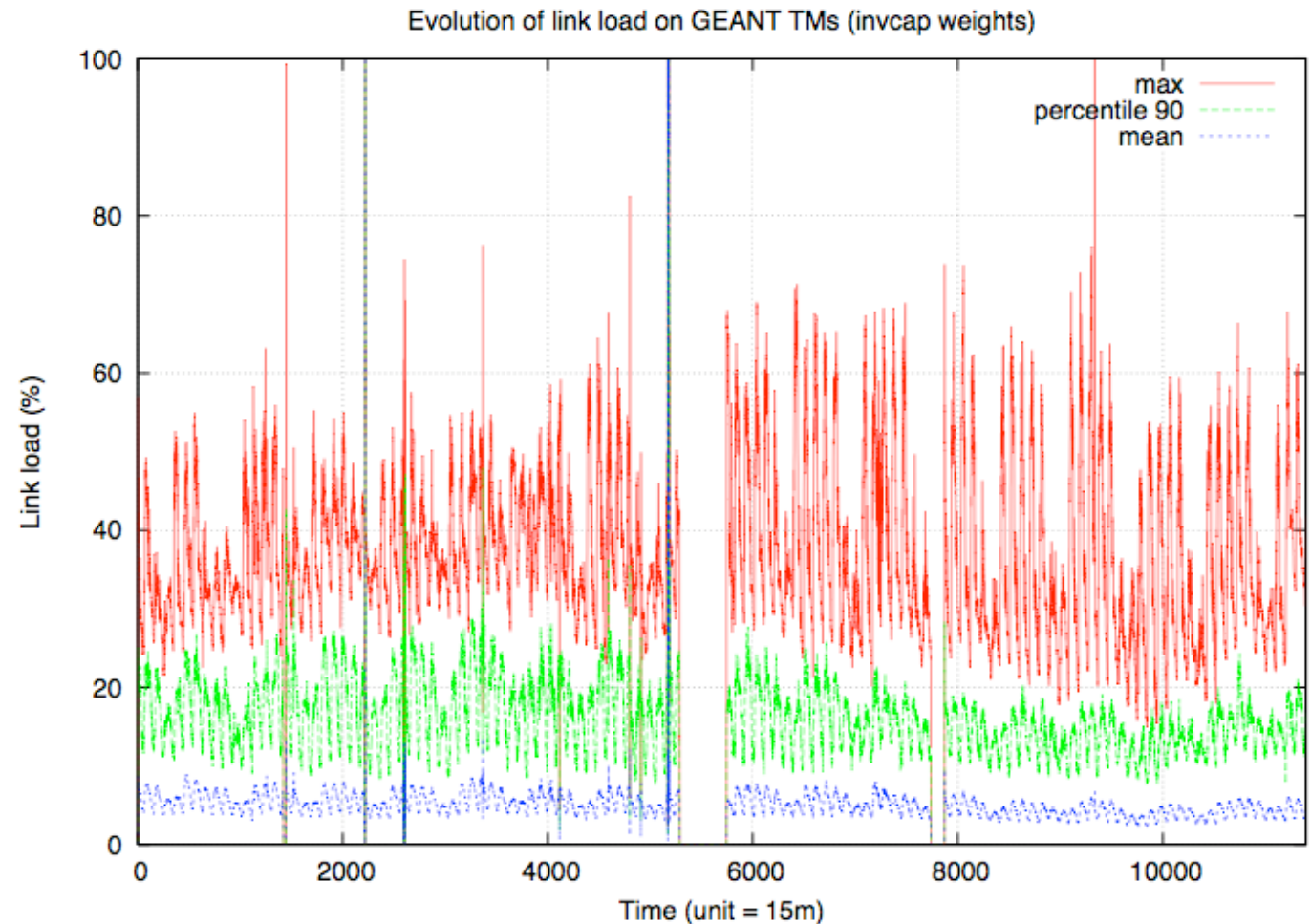
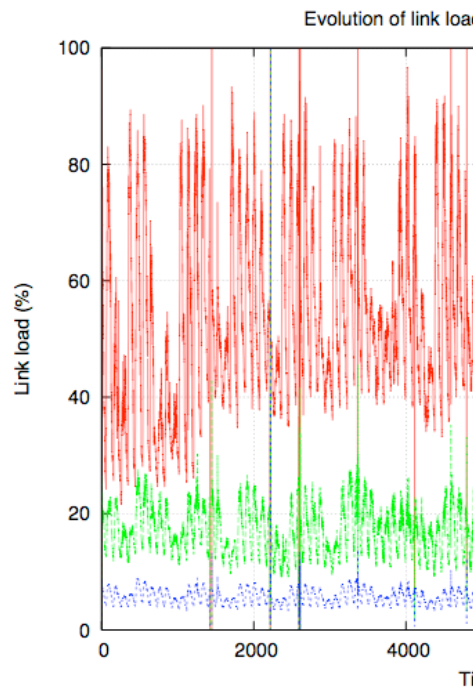
Traffic matrix

A->B : 0.6
A->F : 0.6
B->F : 0.6
A->E : 0.6



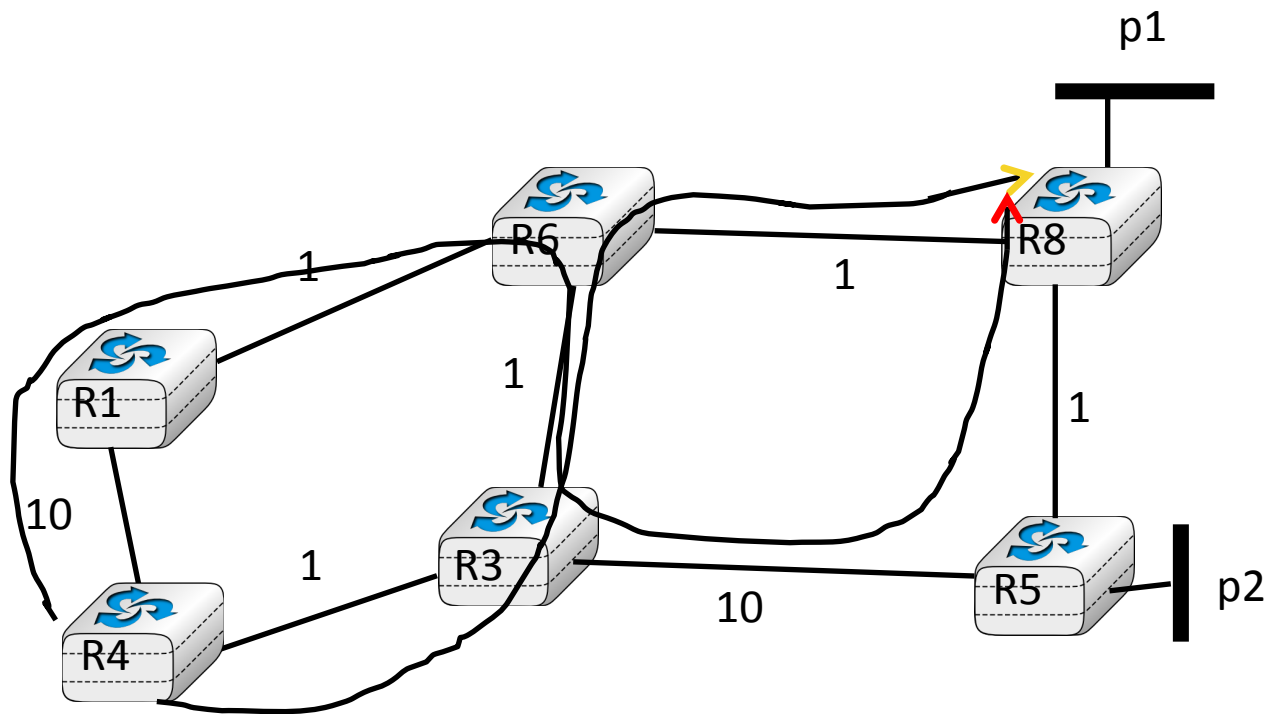
Changing IGP weights in GEANT

- Example : invcap



Traffic engineering

- How to create LSPs along a non-shortest path ?



Traffic engineering

- Two sub-problems
 - How to find a path through the network that meets the requirements for the LSP ?
 - How to create the LSP along this chosen path ?

Requirements for an LSP

- What are the requirements for a traffic engineered LSP ?
 - Ability to reserve x Mbps on all links
 - Maximum propagation delay of y msec
 - Minimise propagation delay
 - Ensure that a new LSP does use the same resources (links, nodes) as an existing one
 -

Traffic engineering extensions to link state routing protocols

- New traffic engineering attributes attached to each link
 - link type and link Id
 - local and remote IP addresses
 - traffic engineering metric
 - additional metric to specify the cost of this link (usually delay)
 - maximum bandwidth
 - maximum amount of bandwidth usable on this link
 - maximum reservable bandwidth
 - maximum amount of bandwidth that can be reserved by LSPs
 - unreserved bandwidth
 - amount of bandwidth that is not yet reserved by LSPs
 - resource class/color
 - can be used to specify the type of link (e.g. Expensive link would be colored in red and cheap links in green)

How to find a specific path through the network ?

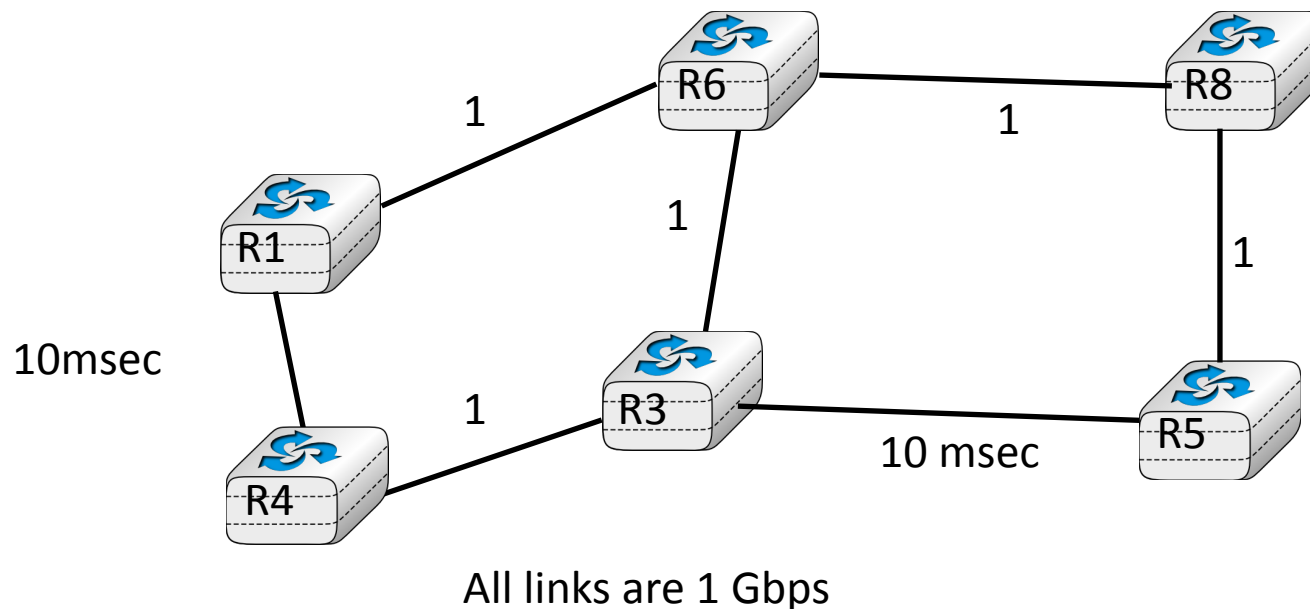
- Path that minimises additive constraint
 - Propagation delay (TE metric)
 - IGP metric
- Dijkstra algorithm
 - Works with only one constraint, in practice either
 - minimize $\text{Sum}(\text{TE metrics})$
 - minimize $\text{Sum}(\text{IGP metrics})$

How to find a constrained path through the network ?

- Examples
 - A path where all links have at least 10 Mbps of available bandwidth
 - A path that has the smallest propagation delay and does not pass through the Atlantic Ocean
 - A path that always passes only through NSA-controlled routers
 - A path that is composed of links having at least 100 Mbps of available bandwidth and does not use satellite links

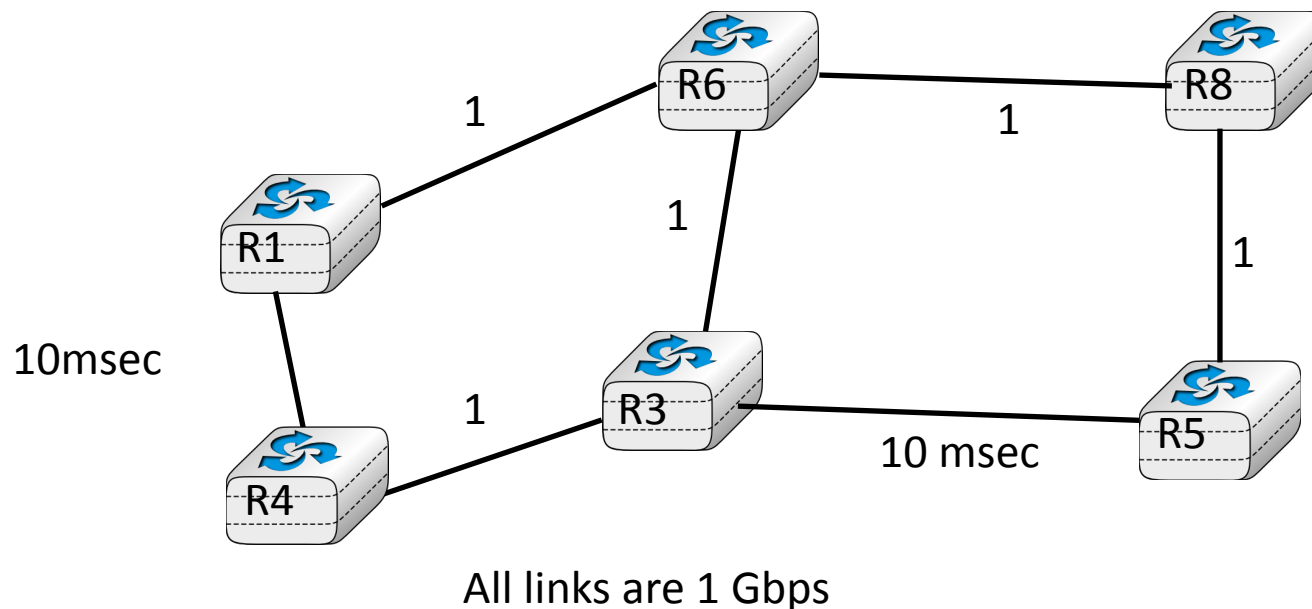
Creating TE LSPs

- TE LSPs
 - R4 -> R5, 600 Mbps, minimum delay
 - R1 -> R5, 600 Mbps, minimum delay
 - R4 -> R8, 300 Mbps, minimum delay



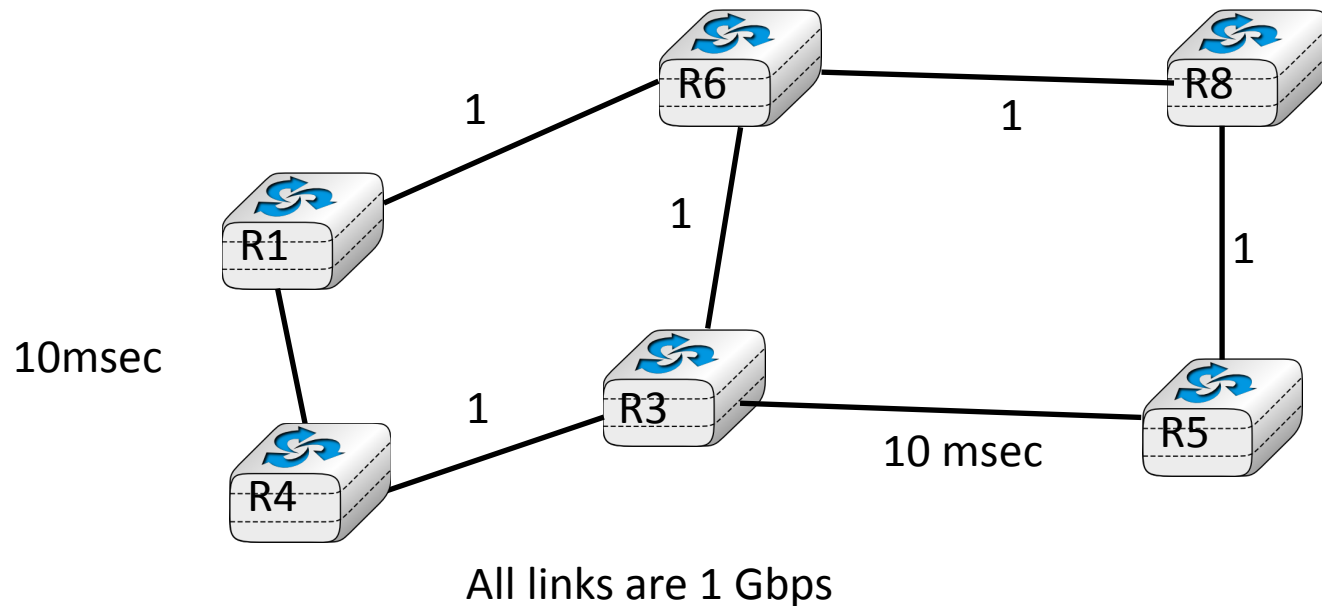
Creating TE LSPs

- TE LSPs
 - R4 -> R5, 600 Mbps, minimum delay
 - R4 -> R5, 600 Mbps, minimum delay, disjoint from previous one



Creating TE LSPs

- TE LSPs
 - R4->R8, 600 Mbps
 - R1->R5, 600 Mbps before having received link state routing update



Issues with TE routing

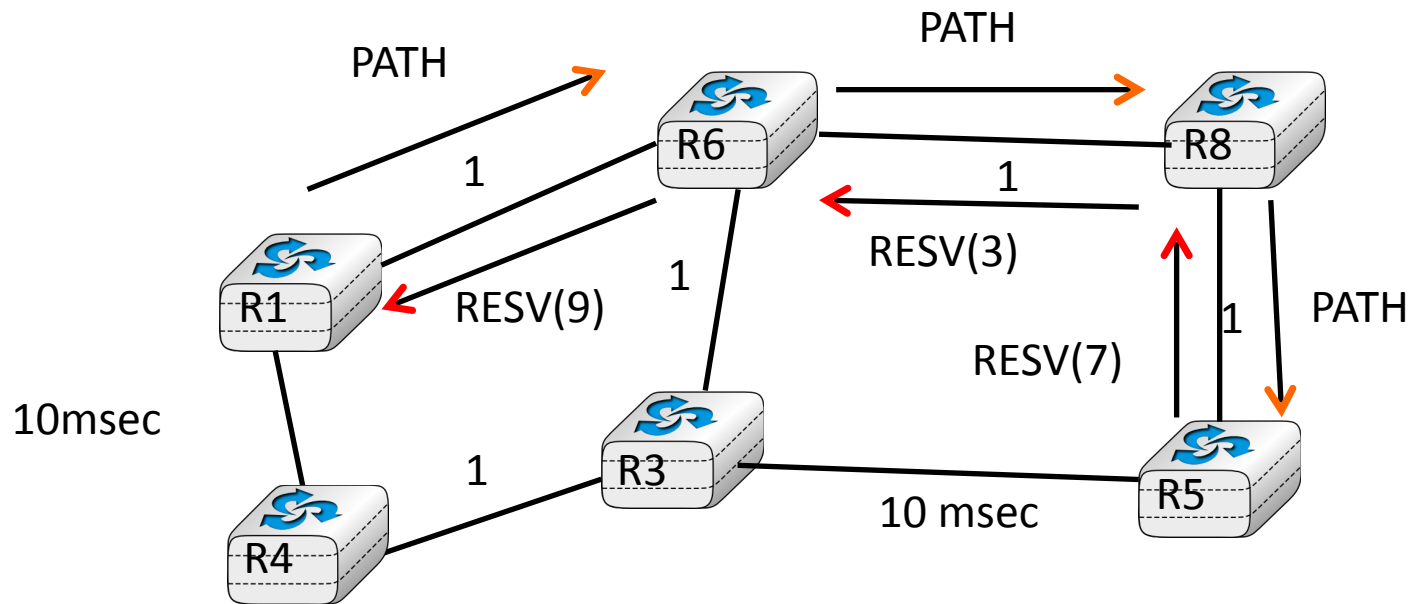
- When to transmit new link state packets ?
 - As soon as one of the TE attributes changes
 - but this could lead to lot of churn in routing
 - Every x seconds
 - There is a time lag between changes in TE attributes and transmission of new link state packet
 - After a significant change in the TE attributes
 - What is a significant change that needs to be quickly advertised ?

Traffic Engineering extensions to link state routing

- Reason for these specific TE attributes
 - link type and link Id
 - local and remote IP addresses
 - traffic engineering metric
 - maximum bandwidth
 - maximum reservable bandwidth
 - unreserved bandwidth
 - resource class/color

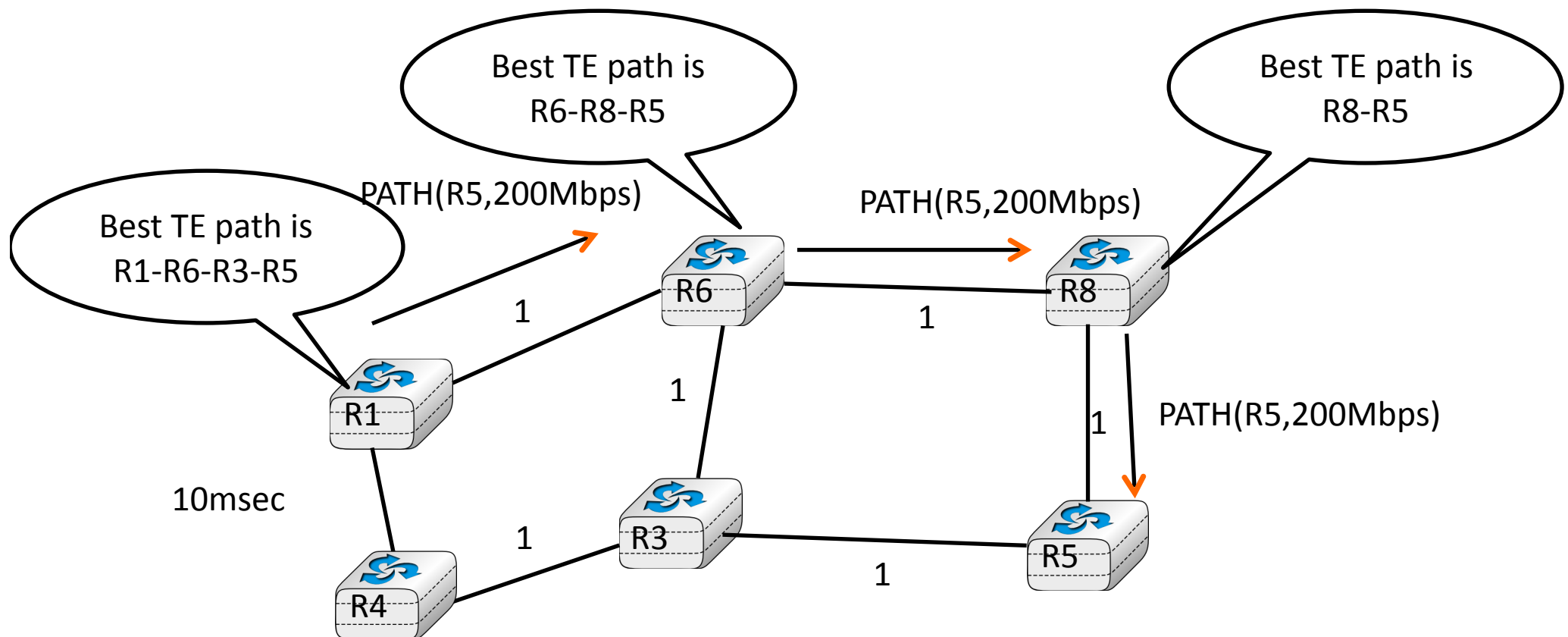
How to create TE LSPs ?

- By using RSVP
 - PATH is used to request a label
 - RESV returns the allocated label



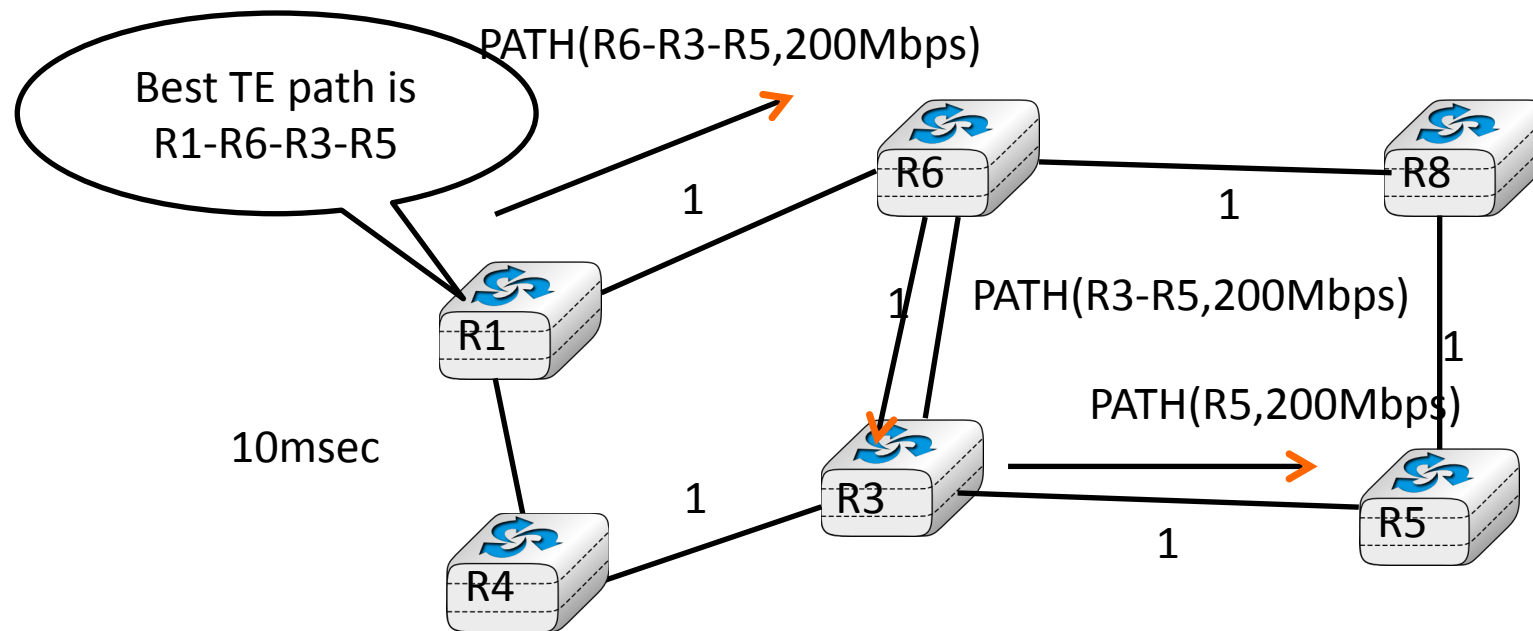
Creating TE LSPs

- How to route PATH message ?
 - Hop-by-hop routing



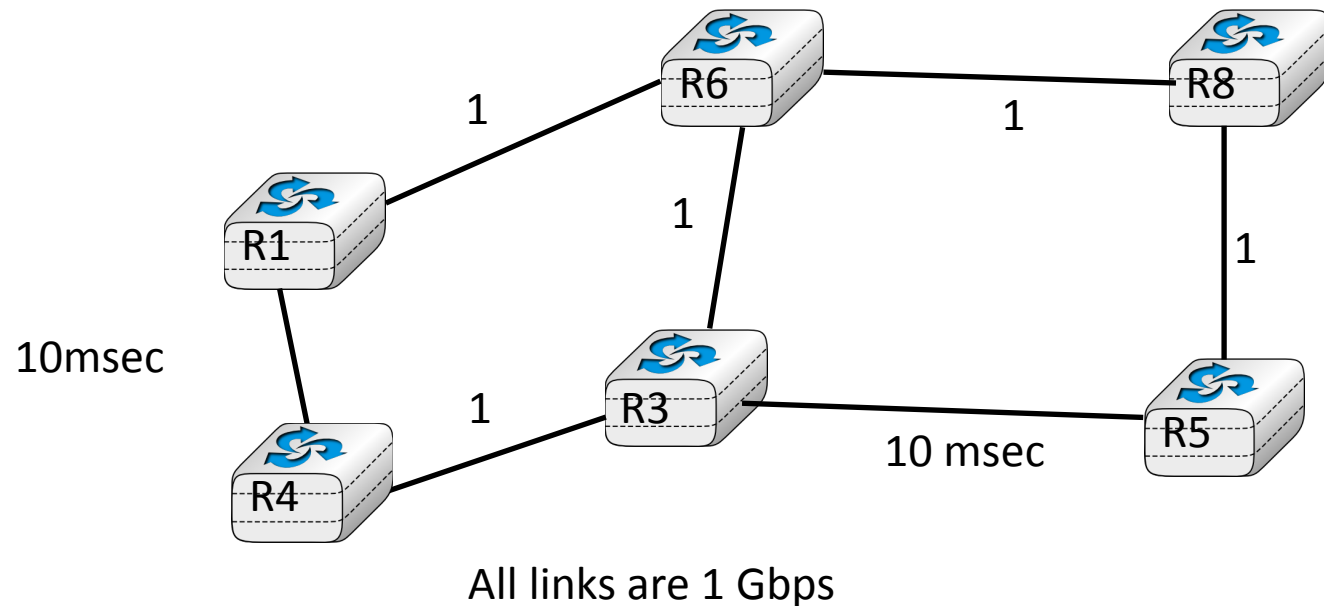
Creating TE LSPs

- How to route PATH message ?
 - Explicitly routing



Creating TE LSPs

- TE LSPs
 - R4->R8, 600 Mbps
 - R1->R5, 600 Mbps before having received link state routing update



Creating LSPs

- What are the advantages and drawbacks of
 - Hop-by-hop routing
 - Explicit routing

RSVP in details

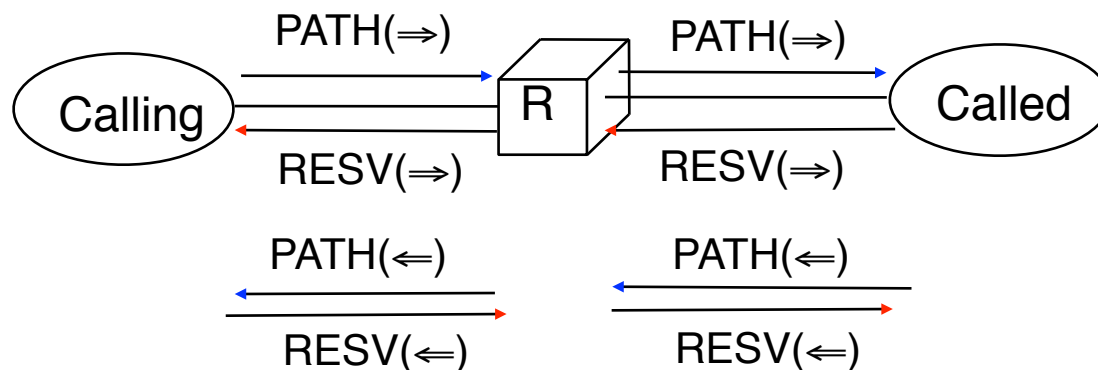
- RSVP : Resource Reservation Protocol
- Objectives
 - Support the establishment of unidirectional flows in IP networks
 - different types of flows
 - initially layer 4 flows
 - today mainly for MPLS LSPs
 - Suitable for IP unicast
 - Suitable for IP multicast

RSVP (2)

- Principles of operation
- Two important RSVP messages
 - PATH
 - used by sender to inform routers and receivers of the new flow and its required resources
 - no resources are reserved due to reception of PATH
 - RESV
 - used by receiver to actually reserve resources for the flow specified in the PATH message
 - resources are reserved for the IP packets sent by the sender towards the receiver along the path taken by the PATH message
- RSVP messages are sent inside IP packets

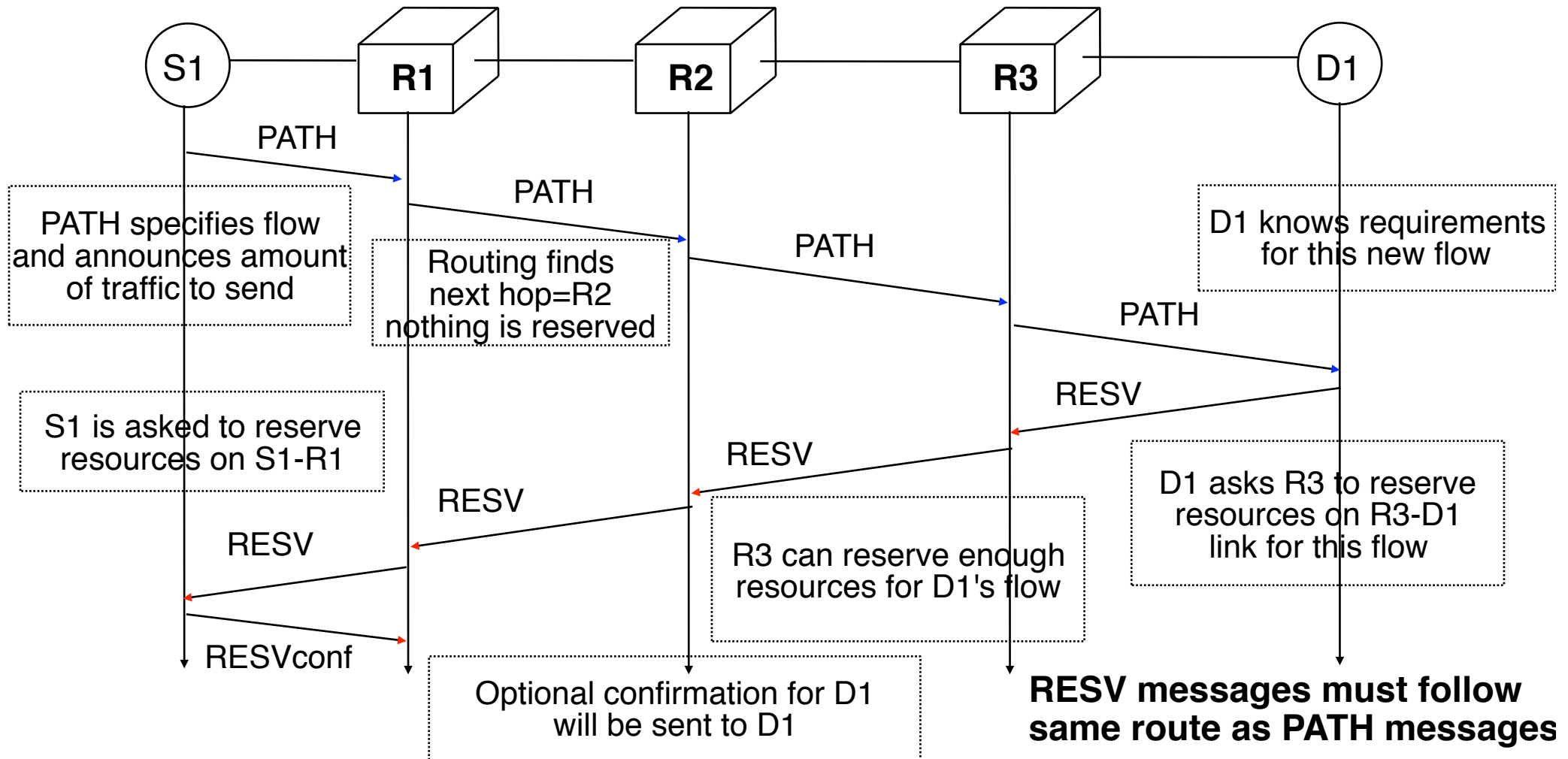
RSVP (3)

- RSVP flows
 - RSVP was originally defined to associate reservations to unidirectional layer 4 flows
 - RSVP flow defined as the set of packets with same
 - IP destination address
 - IP Protocol Number (e.g. TCP or UDP)
 - [Transport-level] Destination Port Number
 - A typical bi-directional voice conversation requires
 - One RSVP flow from calling to called user
 - One RSVP flow from called user to calling



RSVP : example

- Unicast flow establishment with RSVP

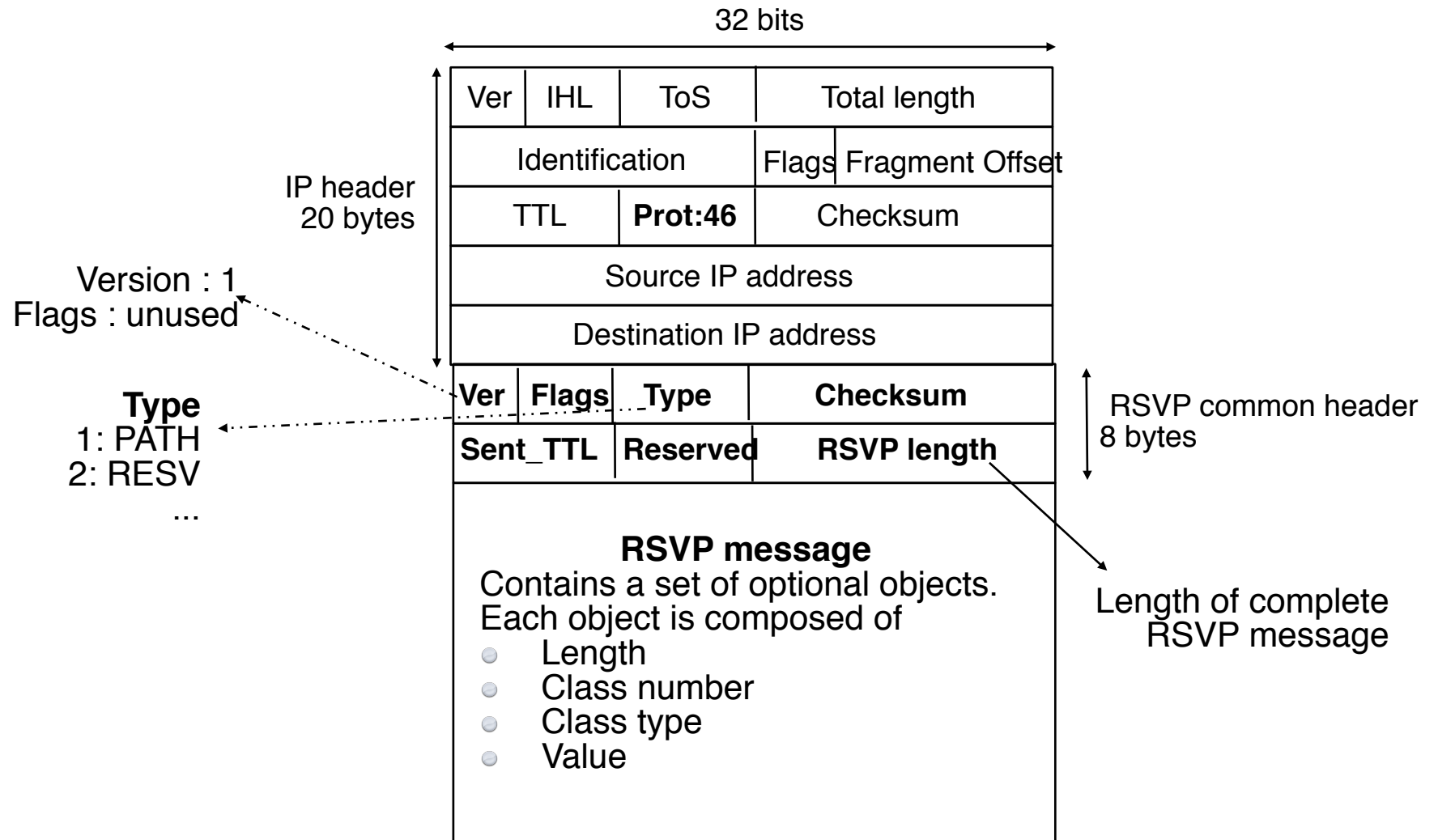


RSVP messages are sent unreliably as simple datagrams

State maintenance

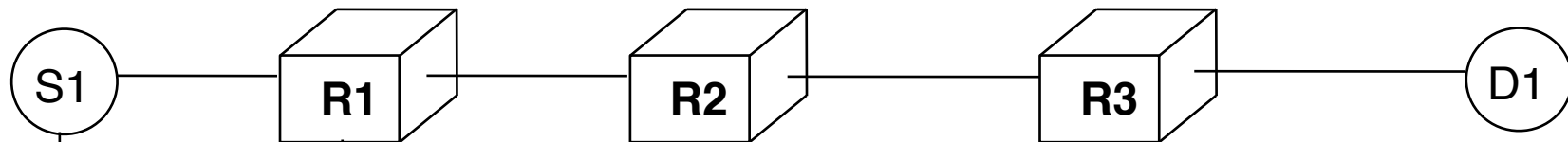
- RSVP routers maintain some per-flow state
- Possible solutions for state maintenance
 - Hard-state
 - traditional solution used in circuit-switched network
 - state is created at flow establishment and removed at flow tear down
 - if intermediate router crashes, state+reservation are lost
 - Soft-state (solution chosen by RSVP)
 - a timer is associated with each per-flow state
 - state is removed upon expiration of timer
 - hosts periodically retransmit PATH and RESV message
 - timer is reset upon reception of PATH/RESV message
 - if intermediate router crashes, state automatically reset
 - if route changes, new reservations/states are established on new path and all reservations/states expires

RSVP : packet format



RSVP : detailed example (1)

- What happens inside the routers ?
 - S1 announces a flow towards D1



Content of PATH message

IP source : S1

IP destination : D1

Previous Hop : S1

Sender Identification : IP addr+ Transport + Port

Destination identification : IP addr + Transport + Port

Timer information (refresh period)

Tspec : X Mbps

R1's state

Flow S1-D1

upstream=S1

Tspec : X Mbps

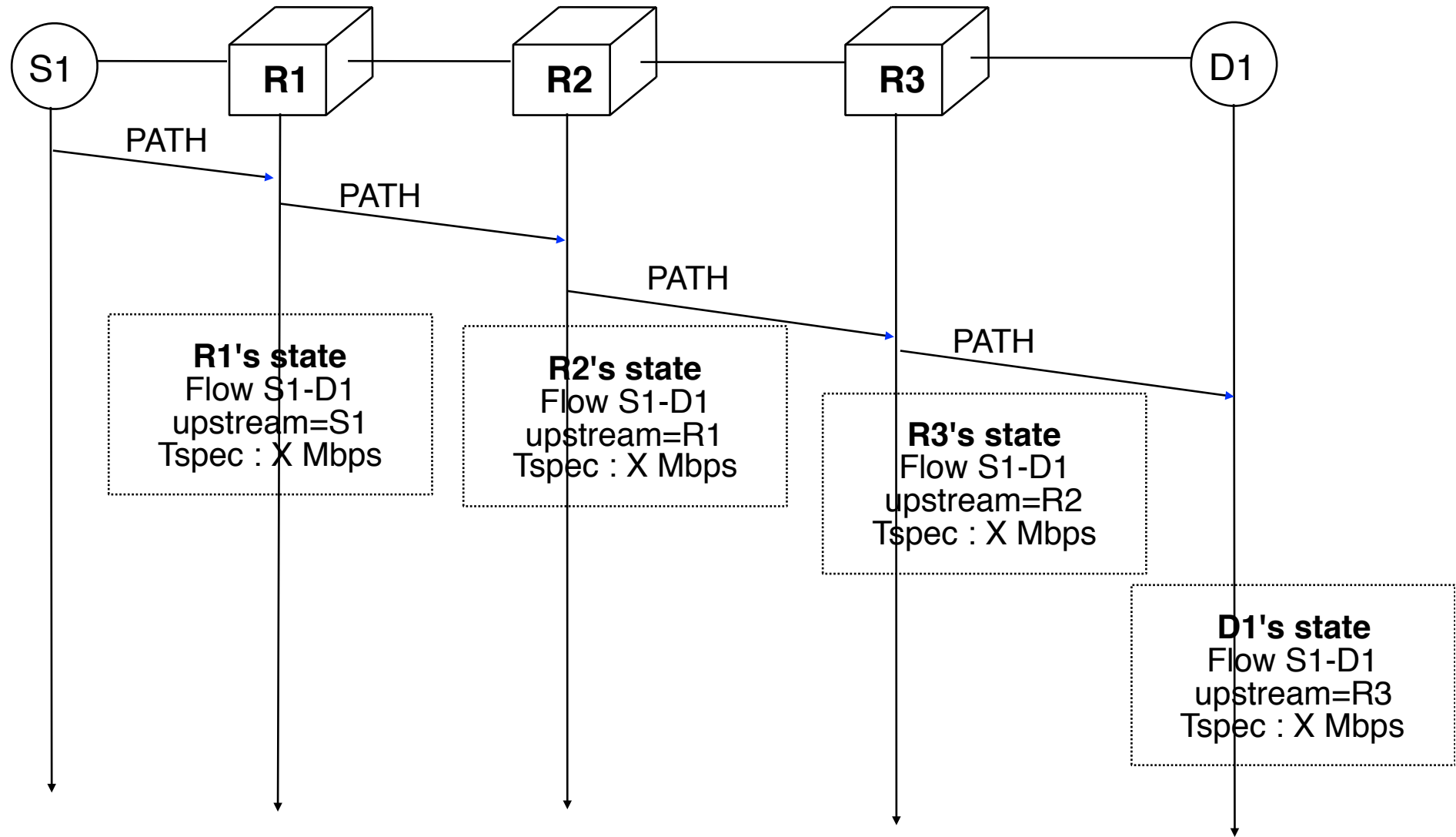
Behaviour of R1

Content of PATH message inserted in R1's

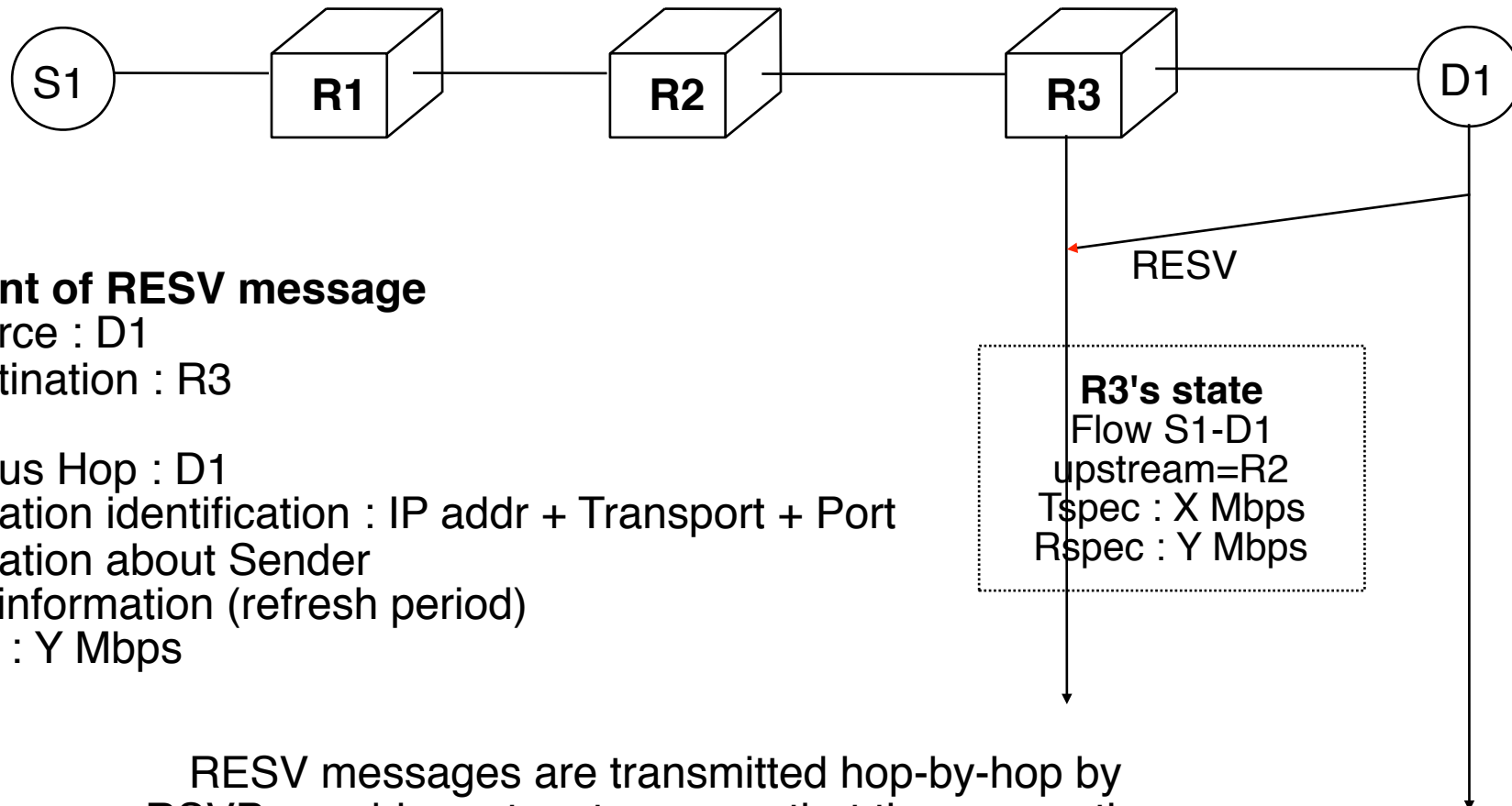
RSVP message updated and sent downstream

PHOP replaced by R1

RSVP : detailed example (2)



RSVP : detailed example (3)



Content of RESV message

IP source : D1

IP destination : R3

Previous Hop : D1

Destination identification : IP addr + Transport + Port

Information about Sender

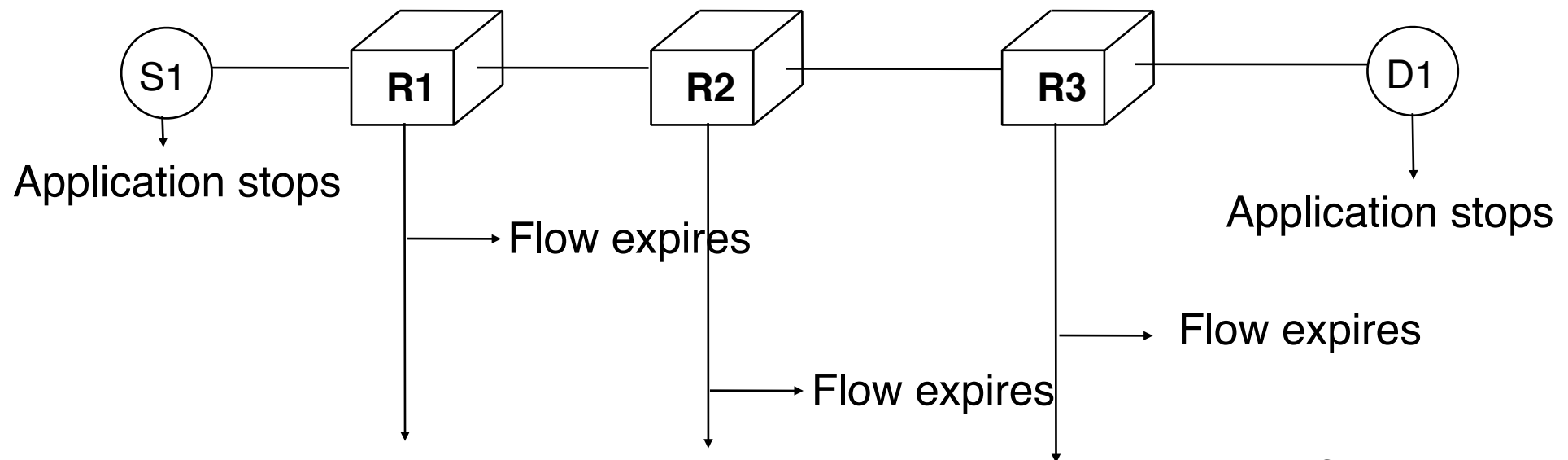
Timer information (refresh period)

Rspec : Y Mbps

RESV messages are transmitted hop-by-hop by RSVP capable routers to ensure that the reservation follows the same route as the corresponding PATH

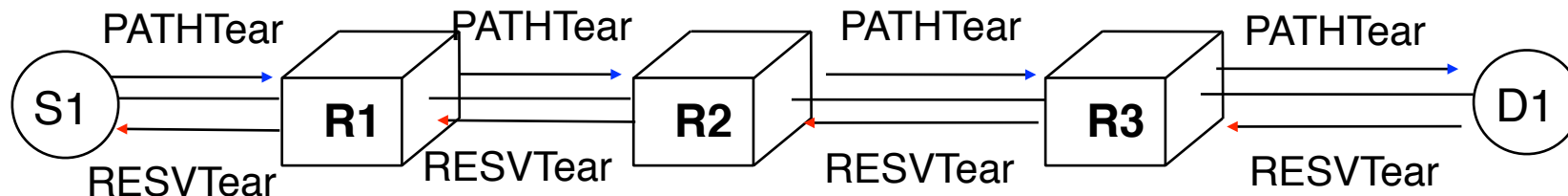
RSVP flow release

- How can we release RSVP flows ?
 - Sender and receiver stopping transmitting PATH and RESV messages
 - flow state will timeout inside routers
 - not the best solution since routers are expected to maintain flow state about five times longer than refresh period without receiving PATH/RESV messages
 - default refresh period is 30 seconds



RSVP flow release (2)

- A better solution
 - Define two new specific signalling messages
 - PATHtear
 - used by sender to announce to network end of a flow
 - RESVtear
 - used by receiver to announce to network end of a reservation



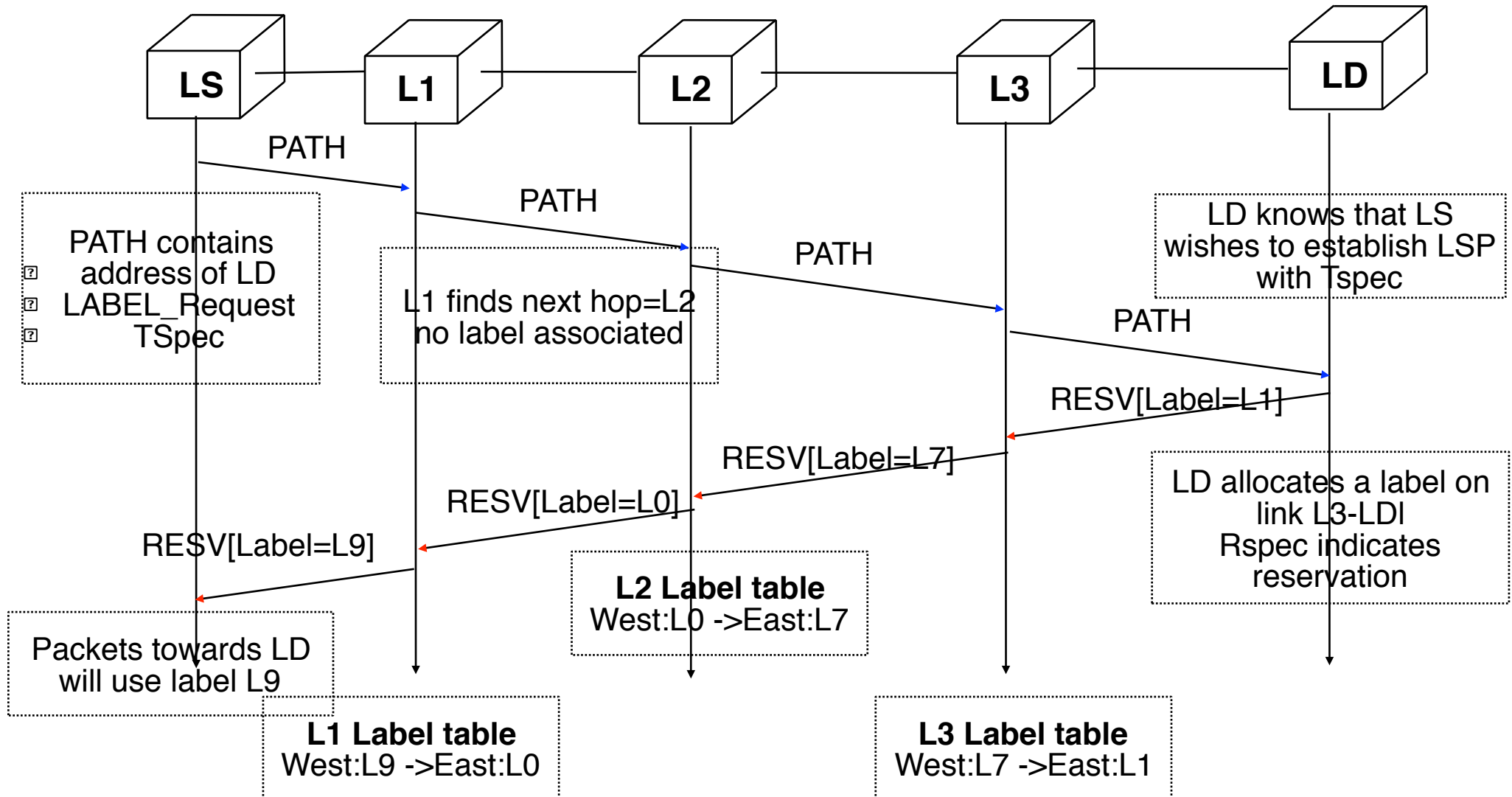
- state timeout remains necessary since all RSVP messages (including PATHtear and RESVtear) are delivered as simple datagrams subject to packet loss

Using RSVP to distribute MPLS labels

- Principle
 - RSVP supports downstream on-demand label allocation
 - RSVP extension for MPLS called RSVP-TE
 - Ingress LSR sends PATH message towards egress LSR
 - PATH message includes Label Request Object
 - Egress LSR sends label back in RESV message
 - RESV propagates the labels hop-by-hop

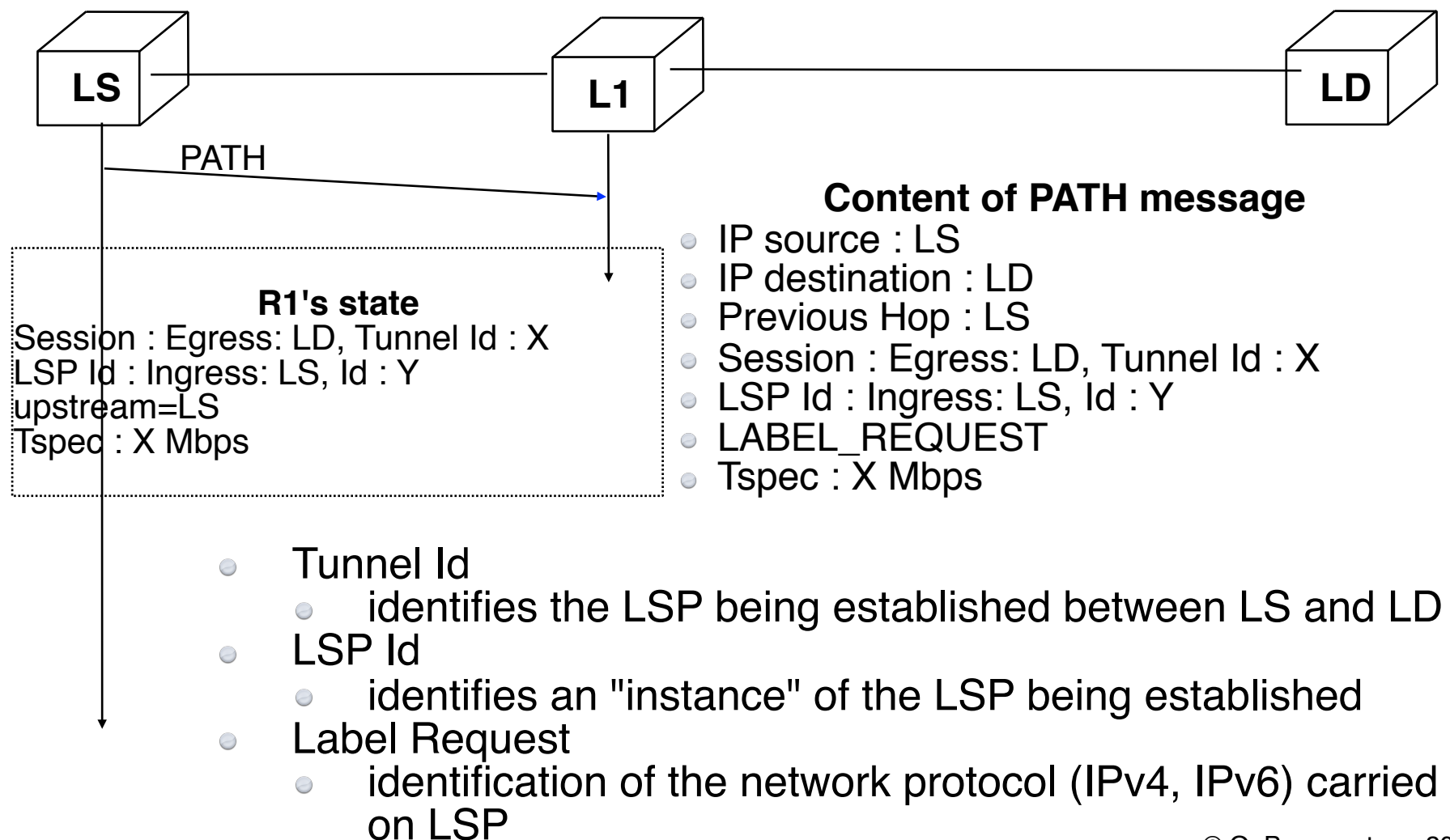
Using RSVP to distribute MPLS labels (2)

- LSP establishment with RSVP-TE



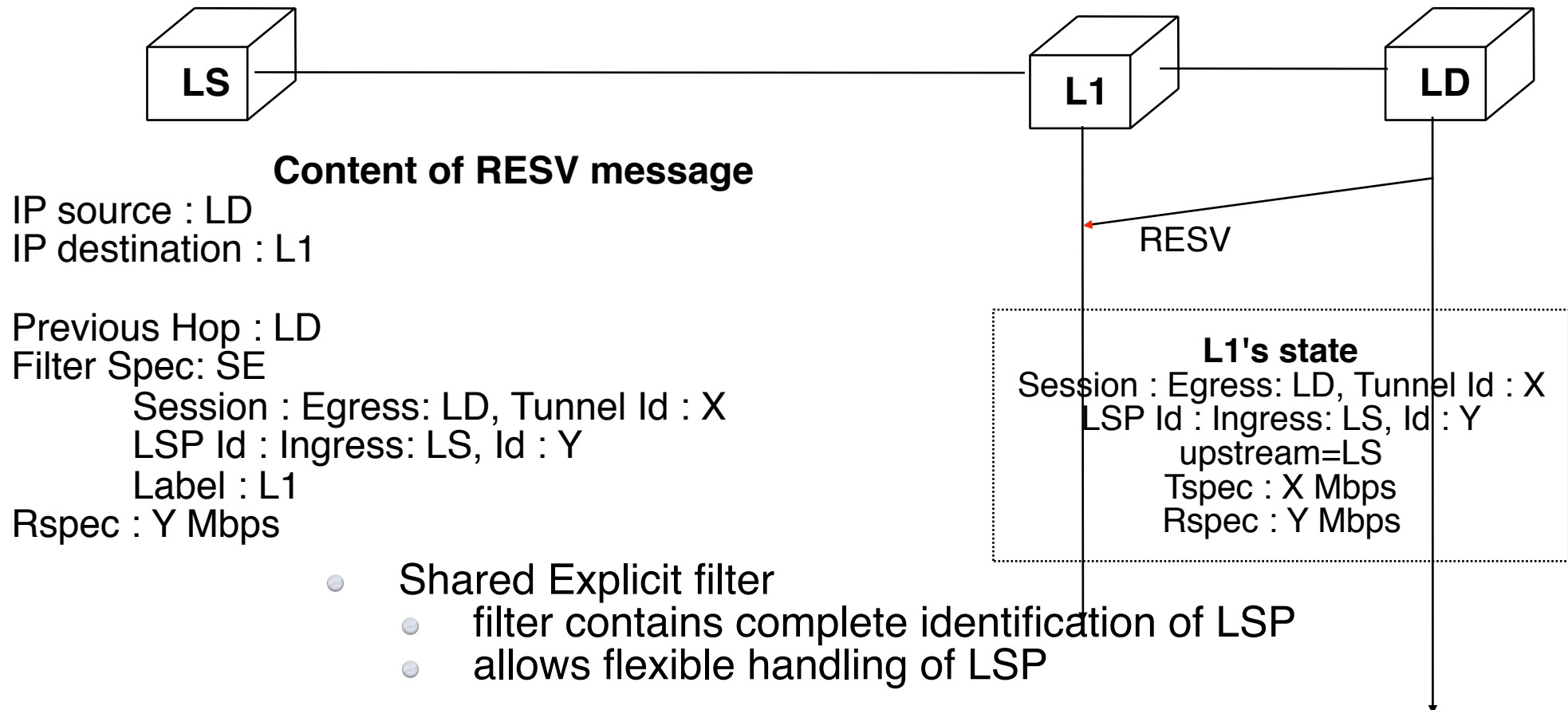
RSVP-TE : detailed example

- What happens inside the LSR ?



RSVP-TE : detailed example (2)

- Content of the RESV message

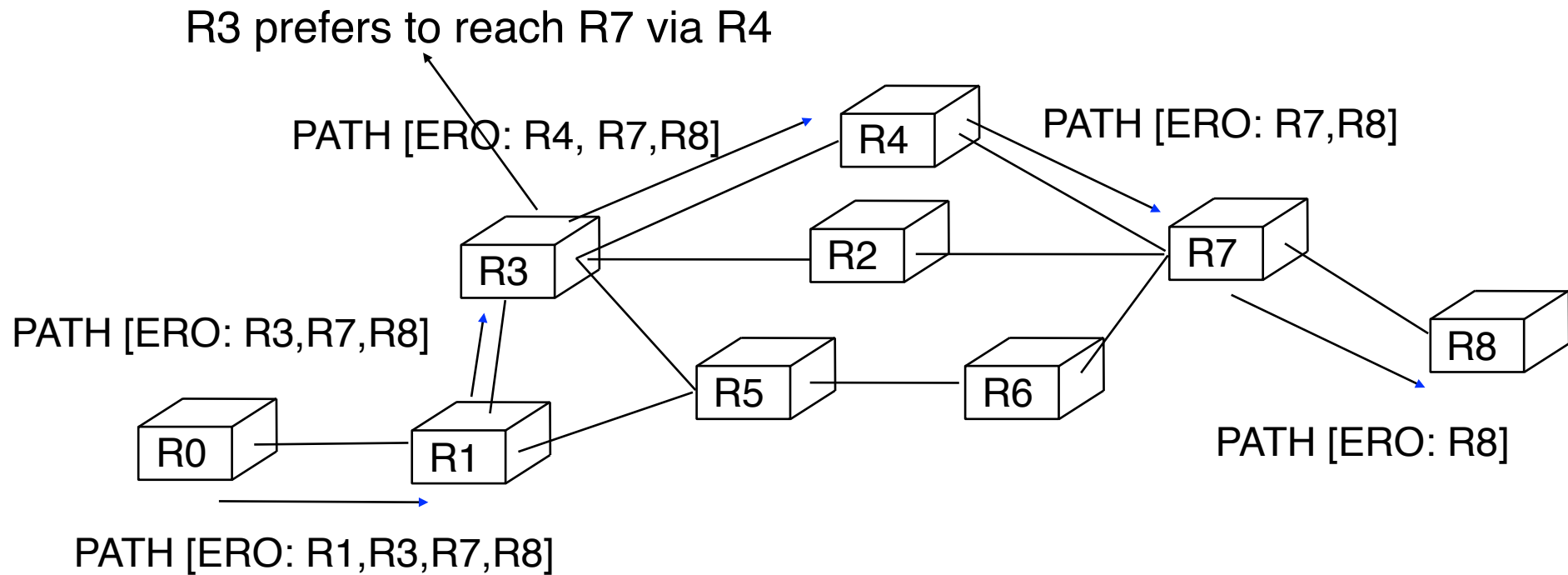


Identification of LSP : [Ingress, Egress, Tunnel ID, LSP Id]

RSVP-TE : Explicit Routes

- How to establish LSP along non-shortest path ?
 - Solution
 - Ingress LSR may specify the route to be followed by an LSP being established
 - Route specification is composed of a list of
 - IP addresses
 - Subnet prefixes
 - Autonomous System numbers
 - Two types of route specifications
 - Strict route
 - the LSP must pass through each LSR specified by ingress LSR
 - Loose route
 - the LSP can pass through non-specified LSR between two specified LSRs

RSVP-TE : Explicit Routes (2)



MPLS for traffic engineering

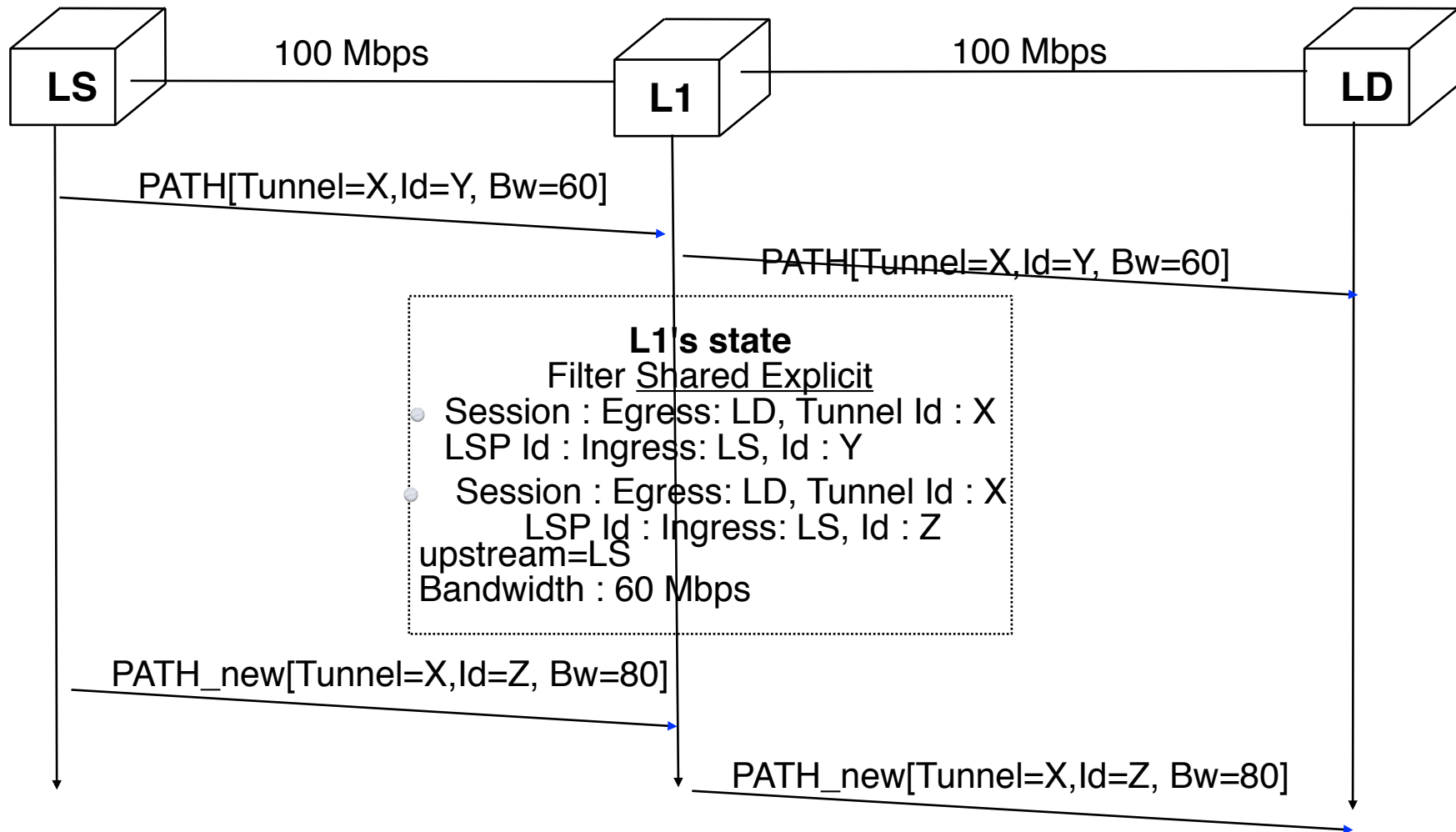
- Principle
 - ingress LSR establishes LSPs (with guaranteed resources) along chosen paths
- Issues to consider
 - How to reserve bandwidth for a given LSP ?
 - Rely on Tspec and Rspec
 - How to support varying traffic flows ?
 - It should be possible to dynamically modify the LSP resources. If there are not enough resources to support an increase, the LSP should keep the old resources
 - How to dynamically reroute an established LSP ?
 - For example more bandwidth is available on another path or because one link used by the LSP failed

RSVP-TE : Resource increase

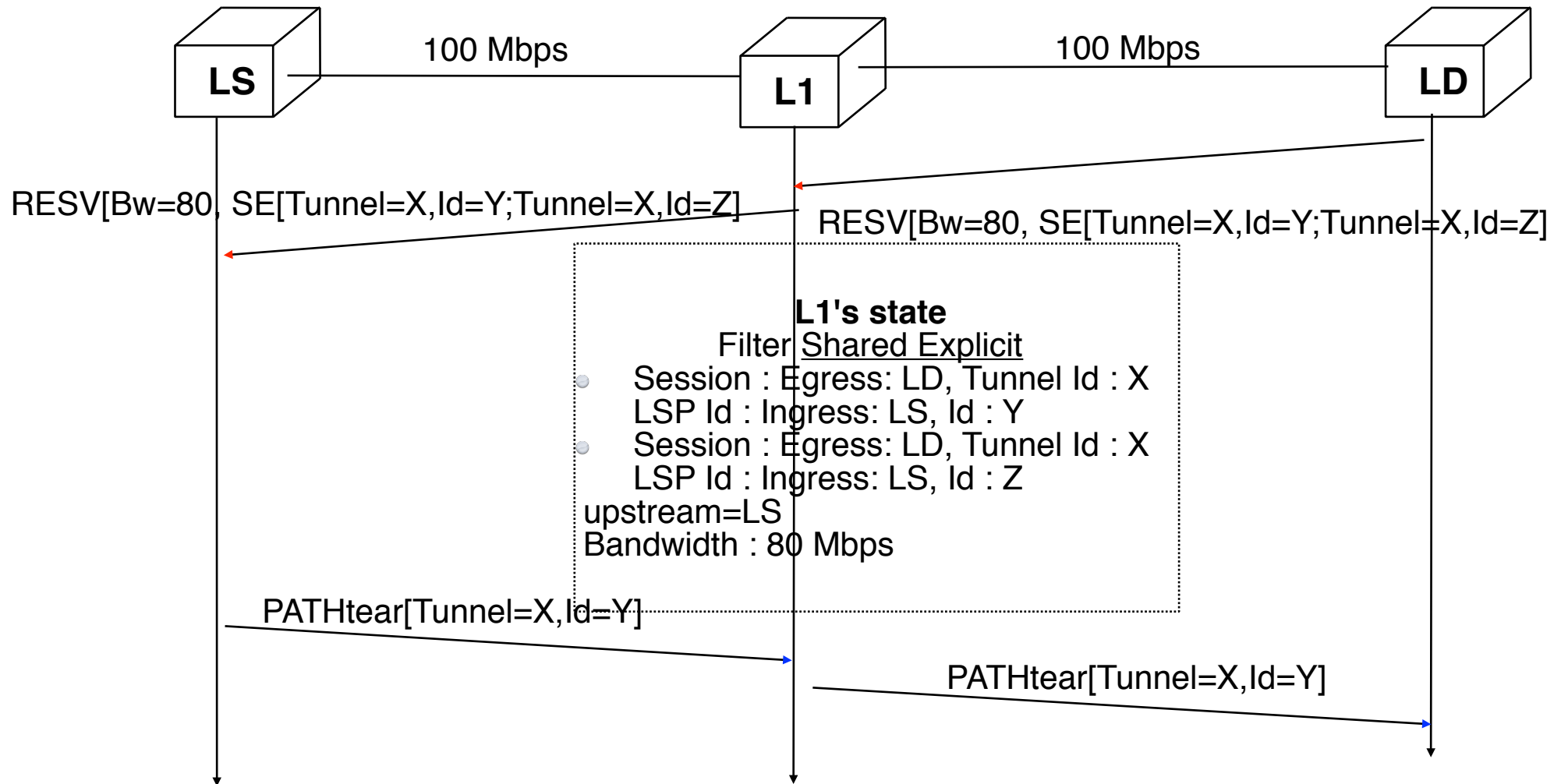
- How to smoothly increase the bandwidth of an LSP ?
 - Simple solutions
 1. Change resources in PATH and RESV messages
 - if there are not enough resources available in the network to support the bandwidth increase, network will send RESVErr and entire LSP will be removed from network
 - not suitable for important LSP
 2. Try to establish new LSP
 - create new LSP and once accepted remove old LSP
 - drawback : the new LSP might be rejected due to the resources already used by the existing LSP

RSVP-TE : Resource increase (2)

- Smooth resource increase

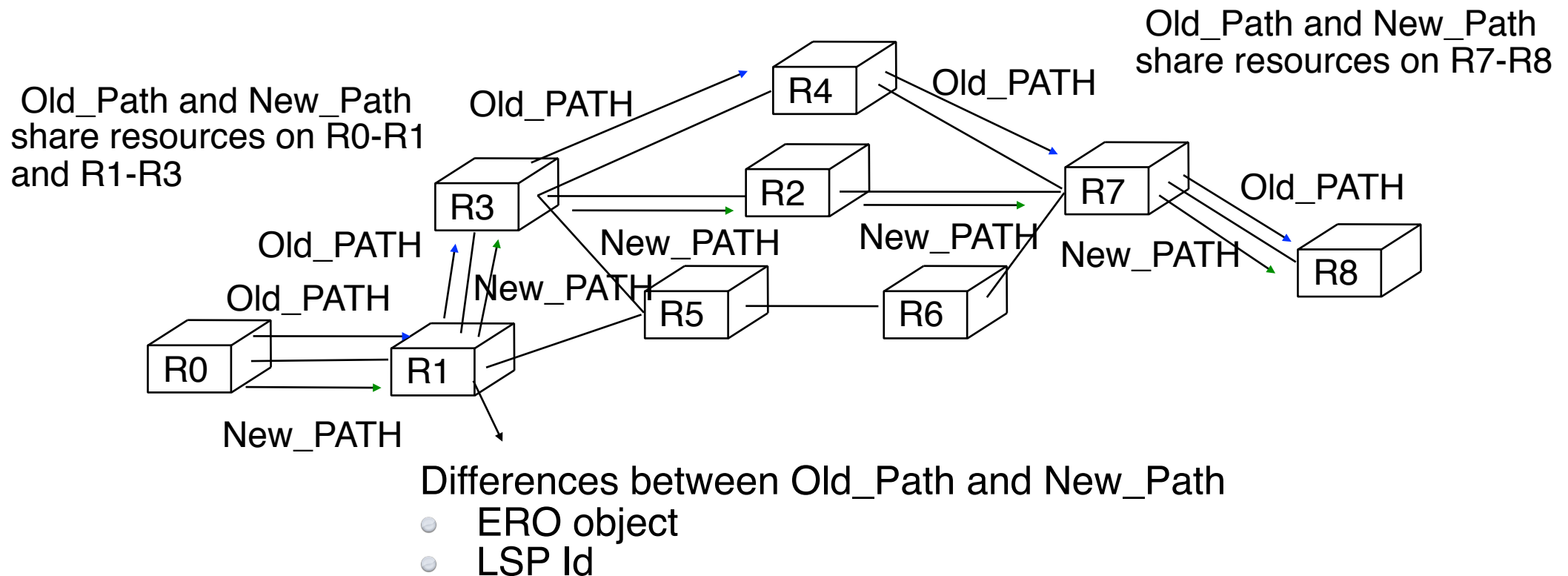


RSVP-TE : Resource increase (3)



RSVP-TE : Changing routes

- How to change the route of an explicitly routed LSP ?
 - Same principle as for resource increase



Traffic engineering with MPLS

A's routing table

F: d=1 : West
D: d=1 : South
B: d=1 : East
G: d=2 : West
C: d=2 : South
E: d=3 : South

A's mapping table

Destination	Label
B	LB
E	LE
F	LF

B's mapping table

Destination	Label
F	LF

B's routing table

A: d=1 : North
C: d=1 : South
D: d=2 : South
E: d=2 : South
F: d=2 : North
G: d=3 : South

Traffic matrix

A->B : 0.6
A->F : 0.6
B->F : 0.6
A->E : 0.6

